Smaller Universal Reversible Turing Machines and the Nonexistence of Two-State Ones

Tetsuo Yokoyama
Nanzan University

October 2026
Extended version

Abstract

A reversible Turing machine (RTM) is a Turing machine in which every instantaneous description has at most one predecessor. Morita's book (2017) lists nine universal RTMs (URTMs); we write URTM(m, n) for one with m states and n symbols. We construct seven URTMs: URTM$(3, 600)$, URTM$(4, 41)$, URTM$(5, 38)$, URTM$(6, 30)$, URTM$(7, 28)$, URTM$(8, 21)$, and URTM$(85, 2)$. Compared with each of the nine, each of them has fewer states or fewer symbols; that is, none of them is *dominated* by any of the nine. Three of Morita's nine are dominated by the new machines, and the remaining six together with the seven new ones give 13 points none of which dominates another. The new machines come from improved versions of Morita's conversions to three states, to four states, and to two symbols. Morita's conversions to three and four states carry the number of a state of the original machine on the tape as a counter. We use disjoint ranges of state numbers by the direction of entry, counting in opposite directions for the two directions, a block decomposition of state numbers that adds a few states to remove symbols, and the merging of symbols that are never read by the same state and never written into the same state. For example, for an original machine with m states and n symbols, the improved conversion to three states uses $O(mn^2)$ symbols instead of $O(m^2n^3)$, and it turns URTM$(32, 3)$ into URTM$(3, 600)$ instead of URTM$(3, 36654)$. On the negative side, we show that the halting problem of RTMs with at most two states is decidable for finite instantaneous descriptions, by a single procedure that also takes the machine as input. Consequently, there is no URTM with at most two states, which answers a question left open by Morita. We also show that no three-state, two-symbol RTM is universal. All these results are formalized in Lean 4 with Mathlib. Universality is formalized after Morita's definition, with computable encoding and decoding functions, which we give explicitly for all 13 points.

1 Introduction

Reversible computing concerns computations in which each step uniquely determines its predecessor. Bennett showed that irreversible computations can be simulated reversibly without loss of computational power [2]. A reversible Turing machine (RTM) is one of the basic models of reversible computing, and a universal RTM is called a URTM. As for ordinary universal Turing machines (UTMs) [15, 7, 13, 16], a natural question is how small a URTM can be. We measure the size by the numbers of states and symbols, and write URTM(m, n) for a URTM with m states and n symbols.

Morita's book [10] lists nine URTMs. Six of them, URTM$(10, 8)$, $(13, 7)$, $(15, 6)$, $(17, 5)$, $(24, 4)$, and $(32, 3)$, simulate cyclic tag systems with halting condition directly; URTM$(17, 5)$ is due to Morita and Yamaguchi. The other three, URTM$(4, 168)$, $(138, 2)$, and $(3, 36654)$, are obtained from them by general conversions that reduce the number of states or of symbols. The book remarks that these three leave room for reduction, and it states that the existence of a two-state URTM is open. In a later survey, after recalling that two-state TMs are universal in

the irreversible case, Morita writes that "it is unknown whether the class of two-state RTMs is universal" [11].

Contributions.

- *New upper bounds* (Section 3). We give seven URTMs, $(3, 600)$, $(4, 41)$, $(5, 38)$, $(6, 30)$, $(7, 28)$, $(8, 21)$, and $(85, 2)$, none of which is dominated by Morita's nine points; that is, compared with each of the nine points, each of them has fewer states or fewer symbols (Figure 1). The three-state machine improves the conversion of Morita's Theorem 5.6, the machines with four to eight states come from a block decomposition of the conversion of Theorem 5.5 followed by symbol merging, and the two-symbol machine optimizes the conversion of Theorem 5.4. The block decomposition gives a sequence of machines with four to nine states in which more states mean fewer symbols (Table 3); no conversion in the book yields five to nine states.
- *Universality in the sense of Morita's definition* (Section 4). Morita's Definition 7.1 asks for total recursive encoding and decoding functions, but the book does not give them for its URTMs. We give computable encodings and decodings for all 13 non-dominated points and prove universality in this sense.
- *Nonexistence for two states* (Section 5). The halting problem of RTMs with at most two states is decidable for finite instantaneous descriptions, uniformly in the machine. Hence no URTM with at most two states exists, for any number of symbols. The key fact is that, in a two-state RTM, the state equals the direction of the last move, so a half-line of the tape can tell the rest of the tape only that the head has come back.
- *Nonexistence for three states and two symbols* (Section 6). No three-state, two-symbol RTM is universal.
- *Formalization* (Section 7). All theorems above are formalized and proved in Lean 4 [8] with Mathlib [9], using only the standard axioms. The formalized machine model reproduces the halting step counts of the published test runs of all nine of Morita's machines.

For each number of states, the constructions give an upper bound on the minimum number of symbols of a URTM. We do not claim that any of the new points is minimal.

Figure 1 shows Morita's nine points, our seven points, and the nonexistence results. The dash-dotted line in Figure 1 bounds the region where UTMs are known not to exist [16]: one-state machines, two-state and three-state two-symbol machines (under a definition that leaves one entry of the table empty for halting), and one-symbol machines. These results concern TMs that need not be reversible, and their treatment of halting and of zero-shifts differs from ours, so we do not use them for URTMs in our sense. The green lines mark our own nonexistence results (Sections 5 and 6), which are stated for the same notion of universality as Theorem 8.

Related work. Small UTMs have been studied for a long time; Woods and Neary survey the results [16]. The non-dominated small UTMs listed in Morita's book include UTM$(2, 18)$ and UTM$(4, 6)$ of Rogozhin [15], UTM$(3, 9)$ of Kudlek and Rogozhin [7], and UTM$(5, 5)$, $(6, 4)$, $(9, 3)$, and $(15, 2)$ of Neary and Woods [13]. These machines need not be reversible, and their conventions on halting and on stationary moves differ from those used here. The URTMs of Morita and his coauthors are collected in [10, Ch. 7], together with the conversions that we improve. Turing machines and universal Turing machines have been formalized in Isabelle/HOL [17], Matita [1], and Coq [6], and Mathlib contains a universal partial recursive function. These formalizations treat general constructions rather than small machines, and none of them is about reversible computation. Closest to our two-state result is Dudenhefner's decision procedure, certified in Coq, for the halting problem of reversible two-counter machines with a particular instruction set;

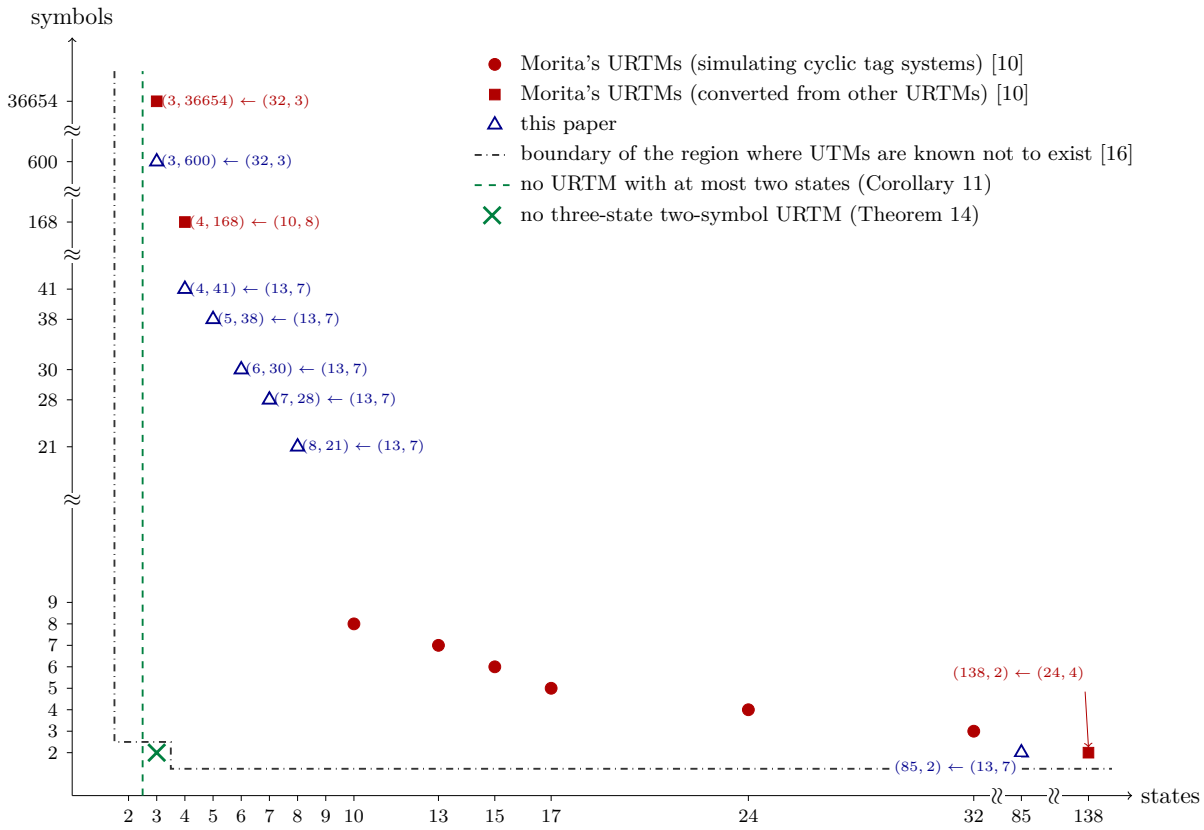


Figure 1: Numbers of states and symbols of URTMs

it contrasts with Morita's result that reversible two-counter machines with another instruction set are universal [5]. To our knowledge, small URTMs and their universality in the sense of Definition 7.1 have not been formalized before.

2 Preliminaries

We follow Morita's book [10] and state the definitions in the form used in the formalization.

Definition 1 (Machines and runs)**.** A *quintuple* $r = [p, s, t, D, q]$ consists of a source state p, a read symbol s, a written symbol t, a direction $D \in \{\mathrm{L}, \mathrm{Z}, \mathrm{R}\}$, and a target state q, where $p, s, t, q \in \mathbb{N}$. The directions L, Z, and R move the head by -1, 0, and $+1$; a quintuple with direction Z is a *zero-shift*. A Turing machine (TM) is a finite list T of quintuples; only membership in T matters. We also call a TM a *table*. No initial or halting states are specified; halting is defined by the absence of an applicable quintuple. An *instantaneous description* (ID) is a triple $(st, pos, tape)$ with $st \in \mathbb{N}$, $pos \in \mathbb{Z}$, and $tape : \mathbb{Z} \to \mathbb{N}$; the symbol 0 is the blank. A quintuple r applies to an ID α if $r.p = \alpha.st$ and $r.s = \alpha.tape(\alpha.pos)$; the result writes $r.t$ at $\alpha.pos$, moves the head by the displacement of $r.D$, and enters $r.q$. An ID is *halting* for T if no quintuple of T applies to it. We write $\mathrm{Halts}_T(\alpha)$ if T reaches a halting ID from α in finitely many (possibly zero) steps.

Definition 2 (Determinism and reversibility)**.** T is *deterministic* if any two quintuples of T with the same source state and read symbol are equal. T is *reversible* if any two distinct quintuples of T with the same target state have the same direction and different written symbols.

This is Morita's Definition 5.4, specialized to the quintuple form. By reversibility, all quintuples entering a state move in the same direction (the unique shift-direction property, [10,

Lemma 5.5]). For an RTM T we write Q_+ and Q_- for the sets of states entered by right moves and by left moves, and $m_+ = |Q_+|$, $m_- = |Q_-|$. A state entered only by zero-shifts, or not entered by any quintuple, belongs to neither set.

Definition 3 (Size)**.** NStates(T) is the number of distinct source states of T, and NSyms(T) is the number of elements of $\{0\} \cup \{r.s, r.t \mid r \in T\}$. A point (m, n) *dominates* (m', n') if $m \le m'$, $n \le n'$, and $(m, n) \ne (m', n')$.

Halting states, which have no outgoing quintuples, are not counted, as in Morita's book (Section 4.2).

Codes of finite IDs. An ID is *finite* if it has finitely many non-blank cells. Let $\mathit{Code} = \mathbb{N} \times \mathrm{List}\,\mathbb{N} \times \mathrm{List}\,\mathbb{N}$. For $p \in \mathbb{Z}$ and $(q, ls, rs) \in \mathit{Code}$, the ID $\mathrm{toID}(p, (q, ls, rs))$ has state q and head position p, carries rs on the cells $p, p+1, \ldots$ and ls on the cells $p-1, p-2, \ldots$, and is blank elsewhere. For an ID α, $\alpha.pos$ is its head position. $\mathrm{IsIDOf}(T, \alpha)$ holds if α is finite, its state occurs in T, and every cell holds the blank or a symbol occurring in T. The enumeration $\mathrm{tm}(i)$, $i \in \mathbb{N}$, decodes i into a list of 5-tuples of natural numbers by the standard bijection in Mathlib, reads each 5-tuple as a quintuple whose direction is L, Z, or R according as the fourth component is 0, 1, or at least 2, and replaces a non-deterministic result by the empty list.

Definition 4 (Universality, after [10, Def. 7.1])**.** A table U satisfies $\mathrm{IsUTMStrict}(U)$ if there are an encoding $c : \mathbb{N} \times \mathit{Code} \to \mathit{Code}$ and a decoding $d : \mathit{Code} \to \mathrm{Option}\ \mathit{Code}$, both computable, such that for all $i \in \mathbb{N}$ and all $a \in \mathit{Code}$ with $\mathrm{IsIDOf}(\mathrm{tm}(i), \mathrm{toID}(0, a))$:

1. $\mathrm{IsIDOf}(U, \mathrm{toID}(0, c(i, a)))$;
2. if $\mathrm{tm}(i)$ reaches a halting ID β from $\mathrm{toID}(0, a)$, then U reaches a halting ID γ from $\mathrm{toID}(0, c(i, a))$, γ has a code (some g with $\mathrm{toID}(\gamma.pos, g) = \gamma$), and every such code g satisfies $d(g) = \mathrm{some}(b)$ for some b with $\mathrm{toID}(\beta.pos, b) = \beta$;
3. if $\mathrm{tm}(i)$ does not halt from $\mathrm{toID}(0, a)$, then U does not halt from $\mathrm{toID}(0, c(i, a))$.

The weaker predicate $\mathrm{IsUTM}(U)$ drops condition 1 and requires the decoding in condition 2 only for some code of γ.

IsUTMStrict implies IsUTM. We use the strong form for existence and the weak form for nonexistence whenever possible, so both kinds of results hold under either form, except the nonexistence of three-state, two-symbol URTMs (Theorem 14), which is stated for the strong form. The requirement in condition 2 that *every* code of γ decodes correctly matters: a code can be padded with trailing blanks without changing the ID it denotes, so a condition on *some* code alone would let the correctness of decoding depend on how the code is written (the number of trailing blanks), which is weak. Morita's IDs are finite strings with the state inserted just before the symbol under the head, without a head position, whereas our IDs are two-way infinite tapes; we prove in Lean that, for deterministic tables, IsUTMStrict implies the formulation on string IDs.

The proofs of universality go through the following cyclic tag systems with halting condition (CTSHs).

Definition 5 (Cyclic tag system with halting condition [10, Def. 7.5])**.** A CTSH is $\hat{C} = (k, (\mathrm{halt}, w_1, \ldots, w_{k-1}))$ with $k \ge 1$ and $w_j \in \{Y, N\}^*$. An ID is a pair (v, j) of a string $v \in \{Y, N\}^*$ and a phase $j \in \{0, \ldots, k-1\}$. The transitions are $(Yv, j) \Rightarrow (vw_j, j+1 \bmod k)$ if $j \ne 0$, and $(Nv, j) \Rightarrow (v, j+1 \bmod k)$ for every j. The IDs $(Yv, 0)$ and (λ, j) are halting, where λ is the empty string.

Morita's six original machines simulate an arbitrary CTSH started from $(v, 0)$, and CTSHs can simulate arbitrary TMs through 2-tag systems [4], [10, Sect. 7.2].

Table 1: Conversions and their effect on the numbers of states and symbols

Conversion	States	Symbols	Example	Source
Reducing states (symbols increase)				
To four states	4	$(2m+1)n$	$(10,8) \to (4,168)$	[10, Thm. 5.5]
Disjoint ranges by direction	4	$(m+1)n$	$(10,8) \to (4,88)$	Sect. 3.1
Reverse counting	4	$\le (\max(m_+, m_-)+1)n$	$(10,8) \to (4,48)$	Sect. 3.1
Blocks and merging of roles	4–9	$n + \sum_{i \in V} \lvert\Sigma_i\rvert$	$(15,6) \to (7,32)$	Sect. 3.2
To three states	3	$N(2M-1)$	$(32,3) \to (3,36654)$	[10, Thm. 5.6]
One symbol per cell, shared counters	3	$O(mn^2)$	$(32,3) \to (3,600)$	Sect. 3.4
Reducing symbols (states increase)				
To two symbols	$\le mk + P_r + P_w$	2	$(24,4) \to (138,2)$	[10, Thm. 5.4]
Code optimization, state merging	per table	2	$(13,7) \to (85,2)$	Sect. 3.5
Same number of states				
Symbol merging	unchanged	number of colors	$(4,50) \to (4,41)$	Sect. 3.3

3 Constructions

All our machines are obtained from Morita's six original machines by conversions. Table 1 summarizes how each conversion changes the numbers of states and symbols; its notation is defined in the respective sections, and an indented row improves the conversion of the row above it. Throughout, T is the RTM to which a conversion is applied and T' is the converted RTM; m, n are the numbers of states and symbols of T, and m', n' those of T'. States of T are written in lower case, such as p, q, p_i, and states of T' in upper case, such as L_0, H_+, A_-. When we distinguish the machines obtained by different conversions, we write $T'_1, T'_2, \ldots$.

3.1 Four states: disjoint ranges by direction and reverse counting

Morita's conversion to four states [10, Sect. 5.3.4] keeps the number of the current state of T on the tape, as a counter attached to the symbol under the head; we write T'_1 for the machine it produces. To execute a quintuple $[p, s, t, D, q]$ of T, T'_1 writes the counter of q into the current cell, places a counter with the initial value in the target cell, and then moves back and forth between the two cells, decrementing the counter of the source cell and incrementing that of the target cell, until the whole number has been carried over. To distinguish the source cell from the target cell, each counter carries an up mark or a down mark. We number the states of T as $p_0, \ldots, p_{m-1}$ and write $[i\uparrow, s]$ and $[i\downarrow, s]$ for the symbol s with a counter of value i and an up or a down mark. We call a symbol of T'_1 made of a symbol of T with a number and a mark a *counter symbol*, and a symbol of T itself a *plain symbol*. There are m values $0, \ldots, m-1$ and two marks, so T'_1 has $(2m+1)n$ symbols: the n plain symbols and $2mn$ marked counter symbols. From URTM$(10, 8)$ it gives URTM$(4, 168)$: with $m = 10$ and $n = 8$, the number of symbols is $n' = (2m+1)n = (2 \cdot 10 + 1) \cdot 8 = 168$. T'_1 has the four states $\mathrm{L}_0, \mathrm{L}_1, \mathrm{R}_0, \mathrm{R}_1$ ($m' = 4$). For a right-moving quintuple $[p_i, s, t, \mathrm{R}, p_j]$ of T, T'_1 uses the rules (named as in Morita's book)

	state of T'_1	read	write	move	next	values of k
$\tilde{\delta}_1, \tilde{\delta}_2$	X_i	$[i\uparrow, s]$	$[j\downarrow, t]$	R	R_0	
$\tilde{\delta}_3$	R_0	s'	$[0\uparrow, s']$	L	L_0	
$\tilde{\delta}_4$	L_0	$[k\downarrow, x]$	$[(k-1)\downarrow, x]$	R	R_1	$1, \ldots, m-1$ ($m-1$ values)
$\tilde{\delta}_5$	R_1	$[k\uparrow, s']$	$[(k+1)\uparrow, s']$	L	L_0	$0, \ldots, m-2$ ($m-1$ values)
$\tilde{\delta}_6$	L_0	$[0\downarrow, x]$	x	R	R_0	

where X_i is L_0 if $p_i \notin Q_+$ (the rule is then $\tilde{\delta}_1$) and R_0 if $p_i \in Q_+$ (the rule is then $\tilde{\delta}_2$); the rules $\tilde{\delta}_3$ to $\tilde{\delta}_6$ do not depend on the quintuple and are given for all symbols s' and x of T. For a left-moving quintuple, the first rule moves left into L_0, and in the other rules L and R are exchanged both in the directions and in the names of the states (L_1 is used only in these left-moving rules). Figure 2 shows how it executes a right-moving quintuple; each row gives the

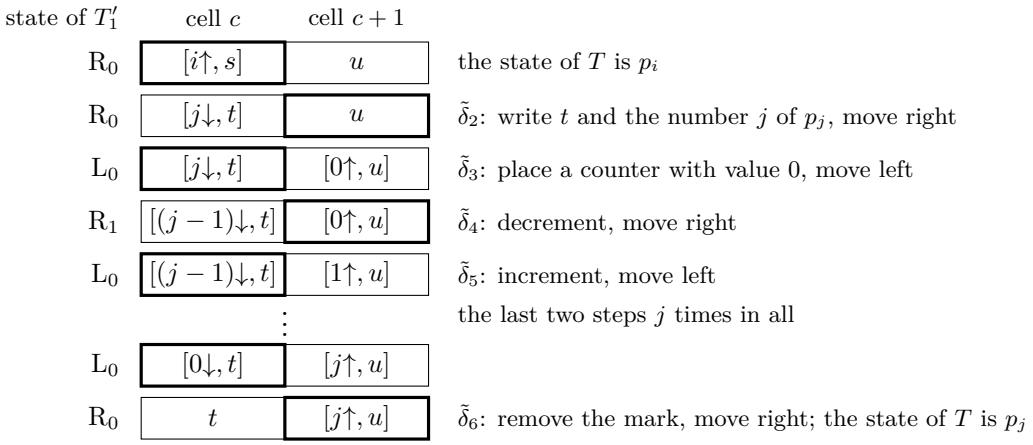


Figure 2: T_1' (Morita's four-state conversion) executing a right-moving quintuple $[p_i, s, t, \mathrm{R}, p_j]$ of T

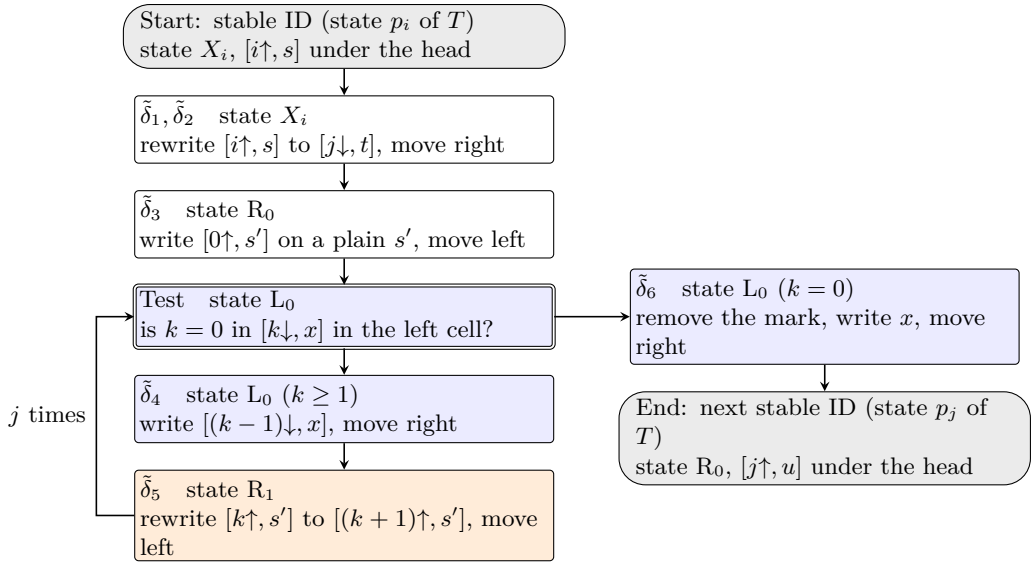


Figure 3: Flowchart of T_1' executing a right-moving quintuple

state and the two cells c and $c+1$ (u is the symbol in cell $c+1$), and the thick frame marks the head. The execution takes $2j+3$ steps. Figure 3 shows the same execution as a flowchart; each box is headed by a rule and the state of T_1' that executes it, and the test in L_0 decides whether the back-and-forth phase continues. We call an ID *stable* if it has exactly one counter symbol on the whole tape, this symbol has an up mark and is under the head, and T_1' is in L_0 or R_0. An ID of T in state p_i with s under the head corresponds to the stable ID of T_1' obtained by replacing that symbol by $[i\uparrow, s]$ and the state by X_i, and a step of T corresponds to the execution of T_1' from a stable ID to the next stable ID [10, proof of Thm. 5.5]. The first and the last rows of Fig. 2 are stable IDs. Figure 4 shows the correspondence with the IDs of the source machine T. Figure 4 also shows the execution of a right-moving quintuple $[p_i, s, t, \mathrm{R}, p_j]$: the thick frame marks the head, the left and right cells are c and $c+1$, and the other cells do not change. To go from the lower row back to the upper row, replace the counter symbol by its plain symbol and take the number of the counter as the state number of T; the head position is kept.

Disjoint ranges by direction. In T_1', L_0 reads both the counter of a stable ID (with an up mark) and the counter on the source side during the back-and-forth phase (with a down mark), and so does R_0; the marks tell these two apart. If the state of stable IDs is separated from the

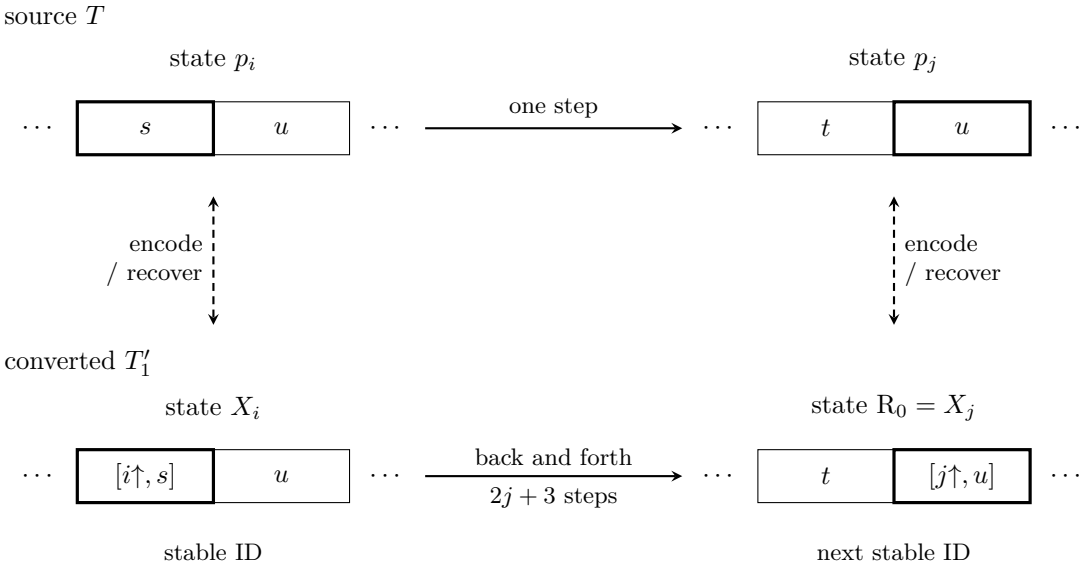


Figure 4: Correspondence between stable IDs and IDs of the source machine T

back-and-forth states, and the two roles of a back-and-forth state are told apart by the counter value, the marks should not be needed. In the following conversions, a state that is not entered by any quintuple is put into Q_+. In Morita's tables to which we apply the conversions, the only zero-shifts are the quintuples into halting states, and the conversions map them separately to halting rules (the halting quintuples of Section 4). Hence every state of T is in exactly one of Q_+ and Q_-, and $m_+ + m_- = m$. Number the states of Q_+ by $1, \ldots, m_+$ and those of Q_- by $m_+ + 1, \ldots, m$. We write T_2' for the machine this conversion produces. T_2' has the four states H_+, H_-, A_+, A_-: H is the state of stable IDs, A is a back-and-forth state, and the index is the direction by which the state is entered. Its symbols are the plain symbols s and the counter symbols $[i, s]$. Here too, a stable ID has exactly one counter symbol on the whole tape, under the head, and T_2' in an H-state; it represents the ID of T obtained by replacing $[i, s]$ by s and taking the state numbered i. While a quintuple is executed, there are two counter symbols, one in the source cell and one in the target cell (Fig. 5). In the table below, ν is the numbering, $[\ell_D, h_D]$ is the range of numbers of Q_D, $\bar{D}$ is the direction opposite to D, and s' and x are arbitrary symbols of T; in subscripts, R is identified with $+$ and L with $-$. For a quintuple $[p, s, t, D, q]$ of T with $p \in Q_e$, T_2' uses the rules

	state of T_2'	read	write	move	next	range of values
θ_1	H_e	$[\nu(p), s]$	$[\nu(q), t]$	D	H_D	
θ_2	H_D	s'	$[\ell_D, s']$	$\bar{D}$	$A_{\bar{D}}$	
θ_3	$A_{\bar{D}}$	$[i, x]$	$[i-1, x]$	D	A_D	$i = \ell_D + 1, \ldots, h_D$ ($h_D - \ell_D$ values)
θ_4	A_D	$[j, s']$	$[j+1, s']$	$\bar{D}$	$A_{\bar{D}}$	$j = \ell_D, \ldots, h_D - 1$ ($h_D - \ell_D$ values)
θ_5	$A_{\bar{D}}$	$[\ell_D, x]$	x	D	H_D	

Each row is a rule scheme, which gives one rule for each quintuple of T, each choice of the symbols s' and x, and each value. We call these schemes $\theta_1, \ldots, \theta_5$; they correspond to Morita's $\tilde{\delta}_1$ and $\tilde{\delta}_2$, $\tilde{\delta}_3$, $\tilde{\delta}_4$, $\tilde{\delta}_5$, and $\tilde{\delta}_6$, respectively. The sum of the two counters is invariant during the back-and-forth phase, and θ_5 fires exactly when the whole number has been carried, so a stable ID is restored after $3 + 2(\nu(q) - \ell_D)$ steps. Figure 5 shows the execution of a right-moving quintuple in the same form as Figure 2, with $i = \nu(p)$ and $j = \nu(q)$. The state A_+ acts as the target side of a right move and as the source side of a left move, but the counter values it reads in the two roles lie in disjoint ranges, so the table is deterministic; reversibility follows by the same separation together with the reversibility of T. Figure 6 shows the flowcharts of a right-moving

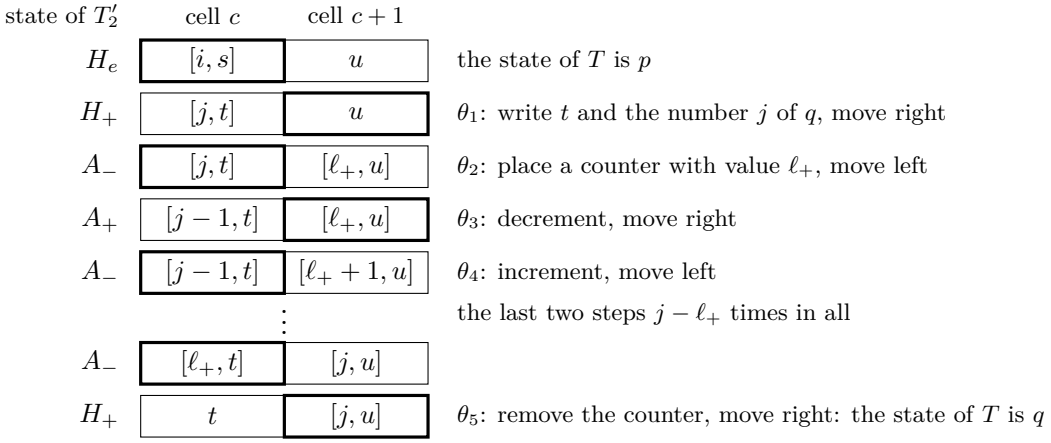


Figure 5: T_2' (disjoint ranges by direction) executing a right-moving quintuple $[p, s, t, \mathrm{R}, q]$ of T

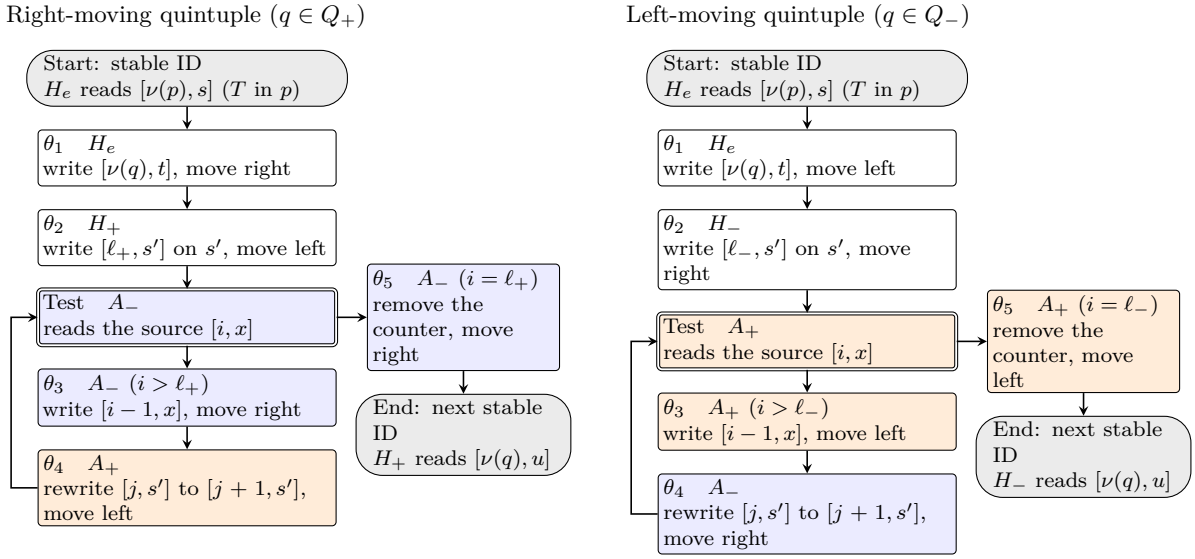


Figure 6: Flowcharts of T_2' executing a right-moving and a left-moving quintuple

and a left-moving quintuple side by side; the shaded boxes are the rules of A_+ and A_-, so the two directions use the same two states and tell the roles apart by the range of the value read. In Morita's conversion, the up and down marks distinguish the source cell from the target cell. Here the subscript of the state (the direction by which the cell was entered) and the range of the counter value make this distinction. Hence the marks are no longer needed, and the number of symbols becomes $(m+1)n$. From URTM$(10, 8)$ this gives URTM$(4, 88)$: with $m = 10$ and $n = 8$, the number of symbols is $n' = (m+1)n = (10+1) \cdot 8 = 88$.

Reverse counting. T_2' uses separate ranges of values for Q_+ and Q_-, but a single carry uses only one of them, so we would like both directions to share one range. If the counters are counted in opposite directions for the two directions of motion (the sentinel, that is, the initial value that θ_2 places and θ_5 detects, is the smallest value for Q_+ and the largest for Q_-), then the decrement rule θ_3 of one direction and the increment rule θ_4 of the other become literally the same quintuple. For example, if Q_- counts downward, θ_3 of Q_+ and θ_4 of Q_- both take the form $[A_-, [i, x], [i-1, x], \mathrm{R}, A_+]$; since they are the same rule, the two roles need not be told apart by ranges of values. The two directions can then share one range of values, and the number of symbols is at most $(\max(m_+, m_-) + 1)n$. We write T_3' for the machine obtained in this way.

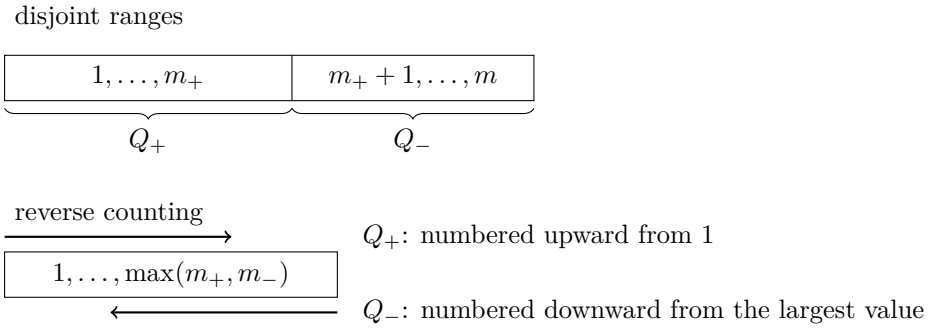


Figure 7: Numbers of the states of T put on counter symbols by T_2' and T_3'

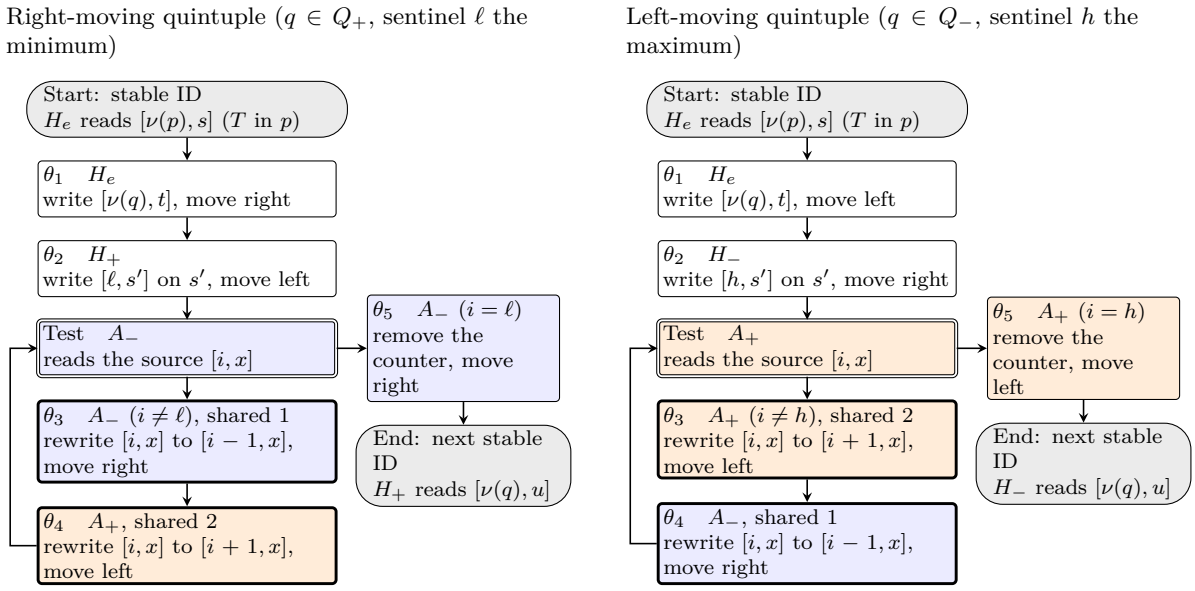


Figure 8: Flowcharts of T_3' executing a right-moving and a left-moving quintuple

Figure 7 compares the two numberings. Figure 8 shows the flowcharts of T_3'; the boxes with thick borders are the rules that are literally the same quintuple for the two directions. From URTM$(10, 8)$ this gives URTM$(4, 48)$: with $m_+ = m_- = 5$ and $n = 8$, the number of symbols is $n' = (\max(m_+, m_-) + 1)n = (5 + 1) \cdot 8 = 48$. We prove that it is universal in the sense of Definition 4, but it is dominated by URTM$(4, 41)$.

3.2 Block decomposition and merging of roles

In T_3' the number of values on the tape still grows with the number of states of T. Moving part of the number into the finite control should increase the states but decrease the values on the tape. Split the number of a state into a pair (block, value within the block), and write T_4' for the machine obtained in this way. The finite control of T_4' holds the block and the tape holds only the value, so a counter symbol $[i, s]$ does not record the block, and different blocks reuse the same values. Let there be b_+ blocks for Q_+ and b_- blocks for Q_-; we write $+k$ for the kth block of Q_+ and $-k$ for the kth block of Q_-. Each block β has three roles: H_β (stable), O_β (the source side of a carry), and G_β (the target side of a carry). Figure 9 shows the splitting for seven states in two blocks. For a quintuple $[p, s, t, D, q]$ of T with $p \in Q_e$, let $\beta = \mathrm{blk}(q)$ be the block of q; then T_4' uses the rules below. A role is a state before merging; the state column gives roles, and the merging of roles described below turns several roles into one state.

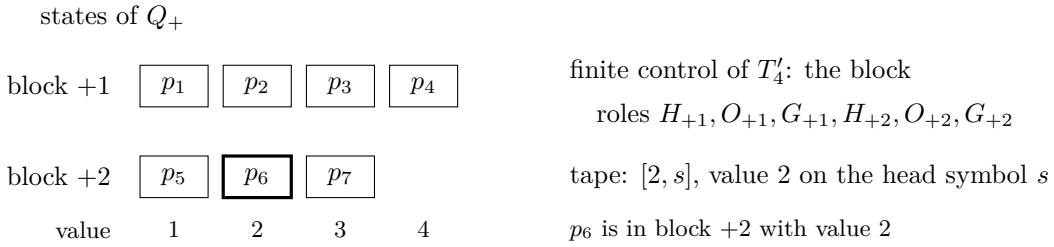


Figure 9: Splitting the numbers of the states of Q_+ of T into blocks and values for T'_4 (an example with seven states)

Table 2: Examples of merging of roles in T'_3 and T'_4

State	Entered	$T'_3 = \text{URTM}(4, 48)$ (one block per direction)	$T'_4 = \text{URTM}(4, 46)$ (two blocks per direction)
H_+	right	H_{+1}	$H_{+1},\ G_{+2},\ O_{-2}$
A_+	right	$G_{+1},\ O_{-1}$	$G_{+1},\ O_{-1},\ H_{+2}$
H_-	left	H_{-1}	$H_{-1},\ O_{+2},\ G_{-2}$
A_-	left	$O_{+1},\ G_{-1}$	$O_{+1},\ G_{-1},\ H_{-2}$

	role	read	write	move	next	range of values
θ_1	$H_{\text{blk}(p)}$	$[\text{val}(p), s]$	$[\text{val}(q), t]$	D	H_β	
θ_2	H_β	s'	$[\sigma_\beta, s']$	$\bar{D}$	O_β	
θ_3	O_β	$[i, x]$	$[i - \varepsilon, x]$	D	G_β	after σ_β up to $\text{val}(q)$ ($\lvert\text{val}(q) - \sigma_\beta\rvert$ values)
θ_4	G_β	$[j, s']$	$[j + \varepsilon, s']$	$\bar{D}$	O_β	from σ_β up to before $\text{val}(q)$ (as many)
θ_5	O_β	$[\sigma_\beta, x]$	x	D	H_β	

Here $\text{val}(q)$ is the value of q within its block, σ_β is the sentinel of block β (the value that θ_2 places and θ_5 detects), and ε is $+1$ if $\text{val}(q) \geq \sigma_\beta$ and -1 otherwise. The back-and-forth phase ends after $3 + 2|\text{val}(q) - \sigma_\beta|$ steps. With one block in each direction, $\text{val} = \nu$, $\sigma_{+1} = \ell_+$, and $\sigma_{-1} = \ell_-$, reading the roles H_{+1}, O_{+1}, G_{+1} as H_+, A_-, A_+ and H_{-1}, O_{-1}, G_{-1} as H_-, A_+, A_- gives the table of T'_2 in Section 3.1. The roles still have to know their block in the back-and-forth phase, because θ_5 must enter the correct H-state; this is what costs states.

With three roles per block as separate states, $3(b_+ + b_-)$ states are needed. Roles that are entered by moves in the same direction and read disjoint values, however, should be able to share one state. A *merging of roles* groups the roles into classes and turns the roles of each class into a single state. Roles in the same class must be entered by moves in the same direction, and the merged table must remain deterministic and reversible. For example, T'_2 and T'_3 of Section 3.1 are the case where all states of Q_+ form one block $+1$ and all states of Q_- form one block -1 ($b_+ = b_- = 1$). Since there is only one block in each direction, all that the finite control holds about the block is the direction, and the number of a state is put on the counter symbol as its value. There are six roles $H_{+1}, O_{+1}, G_{+1}, H_{-1}, O_{-1}, G_{-1}$, so without merging there would be six states. In Fig. 5, A_-, moving back and forth in cell c, plays the role O_{+1}, and A_+, in cell $c + 1$, plays the role G_{+1}. Merging G_{+1} and O_{-1}, both entered by moving right, into one state A_+, and O_{+1} and G_{-1}, both entered by moving left, into A_- gives four states (Table 2). T'_2 tells apart the two roles merged into one state by the ranges of values; T'_3 uses reverse counting to make the rules of the two roles the same quintuples, so that they also share one range. $\text{URTM}(4, 46)$ has two blocks in each direction and hence twelve roles, which are again merged into four classes; blocks ± 1 are the main blocks and blocks ± 2 are blocks with a single state, described next. A further merging that turned out to be useful uses a block with a single state a; we call the block that holds all the other states of the same direction the *main block*. We give a a value outside the range of the main block and make the sentinel of its block the same value; then nothing is

state of T'_4	cell c	cell $c+1$	
$H_{\mathrm{blk}(p)}$	$[i, s]$	u	the state of T is p (value i)
A_+	$[6, t]$	u	θ_1: write t and the value 6 of a, move right; H_a is A_+
H_-	$[6, t]$	$[6, u]$	θ_2: place the sentinel 6, move left; O_a is H_-
A_+	t	$[6, u]$	θ_5: sentinel read, remove the counter, move right; T is in a

Figure 10: T'_4 executing a right-moving quintuple $[p, s, t, \mathrm{R}, a]$ of T into the state a of a single-state block (T is URTM$(10, 8)$ and T'_4 is URTM$(4, 46)$)

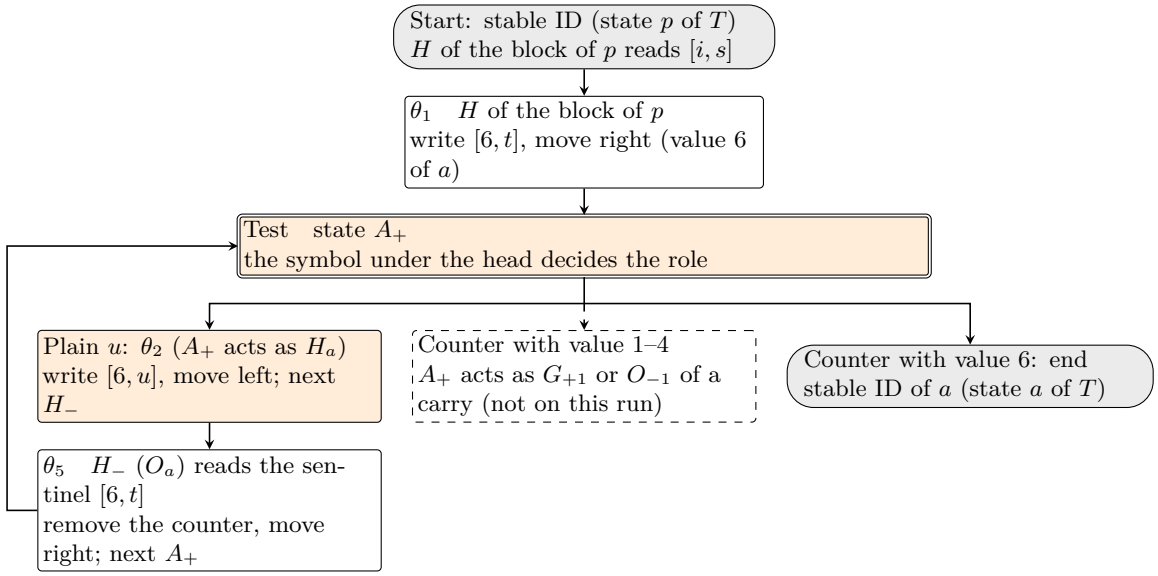


Figure 11: Flowchart of URTM$(4, 46)$ entering the state a of a single-state block

carried on entering a, and there is no back-and-forth phase (three steps, by θ_1, θ_2, and θ_5). We then write H_a, O_a, G_a for the roles of the block of a and merge them into states of the main blocks: H_a into the back-and-forth state entered by moves in the same direction as a (A_+ if $a \in Q_+$), and O_a into the H-state entered by moves in the opposite direction (H_- if $a \in Q_+$). G_a has no rules, so it can be put into any class entered by moves in its direction. Since the back-and-forth state otherwise reads only values in the main range, reading a value outside this range signals that the machine is in a; thus a costs no state of its own. Figure 10 shows the execution in URTM$(4, 46)$ that enters some $a \in Q_+$; the states are named as in Section 3.1, with A_+ the back-and-forth state and H_- the H-state of the main blocks. Figure 11 shows the same execution as a flowchart: the symbol read by A_+ decides whether it acts as the H role of a, as a role of the main block, or as the stable state of a.

Finally, only the counter symbols that can actually occur are kept. If V is the set of counter values used and Σ_i is the set of symbols of T that can carry the value i, the number of symbols is

$$n + \sum_{i \in V} |\Sigma_i| \;\leq\; (|V| + 1)n.$$

The choice of the blocks, of the values, and of the merging is a combinatorial optimization problem. We enumerated partitions of the roles and searched each partition using constraint programming. To explore beyond the enumerated cases, we also used simulated annealing that changes the blocks, counter values, and partitions of the roles. We do not claim optimality of the resulting constructions in general. This gives URTM$(4, 46)$ from $(10, 8)$, for example, with $V = \{1, 2, 3, 4, 6\}$: the main blocks use the values 1 to 4, and two states placed in blocks of

Table 3: Machines obtained by block decomposition and merging of roles

Point	Original	Blocks (b_+, b_-)	V	$\lvert\Sigma_i\rvert$ for $i \in V$	$n + \sum \lvert\Sigma_i\rvert$	$(\lvert V\rvert + 1)n$
$(4, 46)$	$(10, 8)$	$(2, 2)$	$\{1, 2, 3, 4, 6\}$	8, 8, 8, 8, 6	46	48
$(5, 45)$	$(15, 6)$	$(3, 2)$	$\{1, \ldots, 8\}$	3, 5, 5, 5, 5, 5, 5, 6	45	54
$(6, 37)$	$(10, 8)$	$(3, 3)$	$\{1, 2, 5, 6\}$	6, 7, 8, 8	37	40
$(7, 32)$	$(15, 6)$	$(2, 2)$	$\{1, \ldots, 5\}$	6, 5, 5, 5, 5	32	36
$(8, 28)$	$(17, 5)$	$(4, 4)$	$\{2, 4, 5, 6, 7\}$	3, 5, 5, 5, 5	28	30
$(9, 27)$	$(15, 6)$	$(5, 4)$	$\{1, 2, 3, 4\}$	6, 5, 5, 5	27	30

their own share the value 6 (the values were chosen by the optimization). Table 3 lists the six machines with four to nine states obtained in this way. The number of states is the number of classes of roles and need not equal the number of blocks; for example, $(7, 32)$ has blocks $(2, 2)$ and seven states. They are all dominated by the machines of Section 3.3, but they are universal in their own right, and they are the inputs to symbol merging. Morita's book [10] contains no conversion that produces five to nine states.

Remark 6. Three states cannot be reached in this framework, which merges neither two H-roles nor the H- and G-roles of the same block. The former is outside the framework. The latter cannot be merged if the block contains two states that read a common symbol x: if q is the one closer to the sentinel, the H-role reads $[\mathrm{val}(q), x]$ by θ_1 and the G-role reads the same symbol by θ_4, with different actions. If Q_- has at least two states, then either some block of Q_- has two or more states, and its H- and G-roles fall into different classes, or there are two single-state blocks with two H-roles. In either case the roles entered by left moves need at least two classes. The same holds for Q_+, so at least four states are needed for all six of Morita's machines.

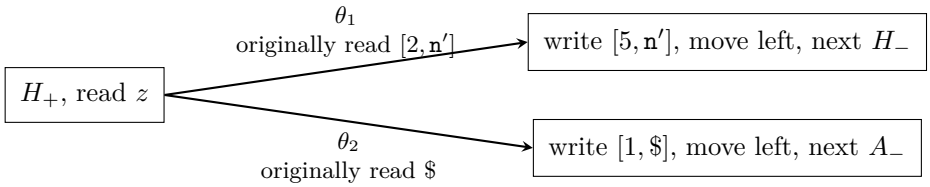
3.3 Symbol merging

In a converted machine T', each counter symbol is read by few states and written into few states. The more states a table has, the fewer states use each symbol and the more pairs of symbols are never read by the same state and never written into the same state, so there should be more room for merging. In Table 4, too, the fraction removed by merging roughly grows with the number of states; this is an observation, not a theorem. UTM$(3, 9)$ of Kudlek and Rogozhin [7] can be read as UTM$(3, 10)$ of Rogozhin [15] with the blank made the same symbol as $\overleftarrow{1}$ and with the encodings of the initial word and of the halting symbol changed accordingly. We apply systematically this idea of reducing symbols by making several of them one symbol. Two symbols x and y *conflict* if some state reads both, some state is entered by quintuples writing x and y, or both can occur on the tape of a halting ID reached from an encoded input. The *conflict graph* has the symbols as vertices and an edge between any two conflicting symbols. Merging pairwise non-conflicting symbols is a coloring of the conflict graph (an assignment of colors in which adjacent vertices get different colors), and symbols of the same color become one symbol. The merged table is deterministic and reversible if the unmerged one is, and it follows the run of the unmerged one step by step.

For example, consider T'_3 obtained from URTM$(10, 8)$ (with the symbols $\mathtt{b}, \mathtt{y}, \mathtt{n}, \mathtt{n}', \mathtt{Y}, \mathtt{N}, *, \$$), that is, URTM$(4, 48)$. The state H_+ reads $[2, \mathtt{n}']$ by θ_1 and the plain symbol \$ by θ_2, so these two conflict (merged, the rule of H_+ would not be determined). The state H_- is entered by the θ_1 rule $[H_+, [4, \mathtt{n}'], [1, \mathtt{n}'], \mathrm{L}, H_-]$, which writes $[1, \mathtt{n}']$, and by the θ_5 rule $[A_+, [5, \$], \$, \mathrm{L}, H_-]$, which writes \$, so these two also conflict (merged, the previous ID of H_- would not be determined). The plain symbols $\mathtt{y}$ and $\mathtt{n}$ conflict because both can occur on a halting tape. Of the 1128 pairs of the 48 symbols, only $\mathtt{n}'$ and $[3, \mathtt{n}]$ do not conflict. The symbol $\mathtt{n}'$ is read only by H_+ and written only by quintuples entering H_-; the symbol $[3, \mathtt{n}]$ is read only by A_+ and A_- and written only by quintuples entering H_+, A_+, and A_-; and counter symbols do not occur on a halting

(a) Conflict between symbols read by the same state

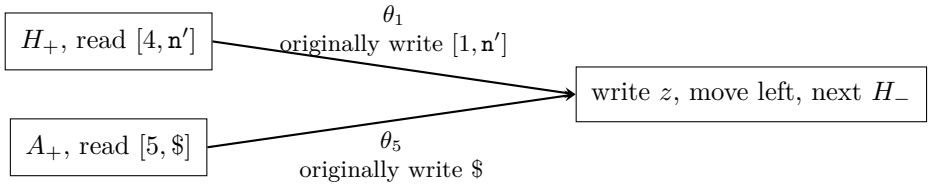
Merge the counter symbol $[2, \mathtt{n}']$ and the plain symbol \$ into z



The same state and read symbol allow two different actions

(b) Conflict between symbols written on entry to the same state

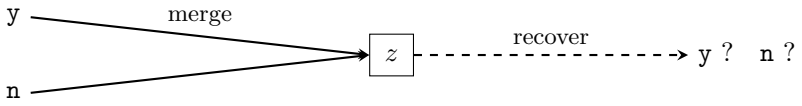
Merge the counter symbol $[1, \mathtt{n}']$ and the plain symbol \$ into z



The same written symbol and next state allow two different predecessors

(c) Conflict between symbols to be recovered after halting

Merge the plain symbols `y` and `n` into z



The symbol z cannot distinguish `y` from `n`

Figure 12: Examples of conflicts under symbol merging

tape. With so few states and uniform back-and-forth rules, almost all pairs conflict. Figure 12 illustrates these conflicts. Each panel is a separate example merging only the indicated pair of symbols into z; arrows show the merged rules or maps on symbols.

The condition for a conflict can be relaxed as follows. If two quintuples from the same state read x and y, have the same direction and the same target state, and write two symbols that are also merged into one, then after merging the two quintuples become the same quintuple. The same holds for two quintuples that enter the same state writing x and y if they have the same source state and the same direction and read two symbols that are also merged into one. Such a pair need not be treated as a conflict, since the two equal quintuples are kept as one. The counts in the example above are for the condition before relaxing.

The following general statement is not taken from the literature; we prove it here.

Theorem 7 (Symbol merging)**.** *Let U satisfy* IsUTMStrict(U)*, and assume that every halting ID reached from an encoded input is in a state that is not a source state and has all its tape symbols in a fixed finite set F. Let $h, \rho : \mathbb{N} \to \mathbb{N}$ be primitive recursive with $h(0) = \rho(0) = 0$ and $\rho(h(x)) = x$ for every $x \in F$. If the table $h(U)$, obtained by replacing the read and written symbols of every quintuple by their images under h, is deterministic, then* IsUTMStrict$(h(U))$*.*

Here h is the map of the coloring, which sends each symbol to its color, that is, to the

Table 4: Machines obtained by symbol merging

Point	Original	Blocks (b_+, b_-)	Symbols before merging	Symbols after merging
$(4, 41)$	$(13, 7)$	$(2, 2)$	50	41
$(5, 38)$	$(13, 7)$	$(3, 2)$	49	38
$(6, 30)$	$(13, 7)$	$(3, 3)$	54	30
$(7, 28)$	$(13, 7)$	$(4, 3)$	50	28
$(8, 21)$	$(13, 7)$	$(4, 4)$	40	21

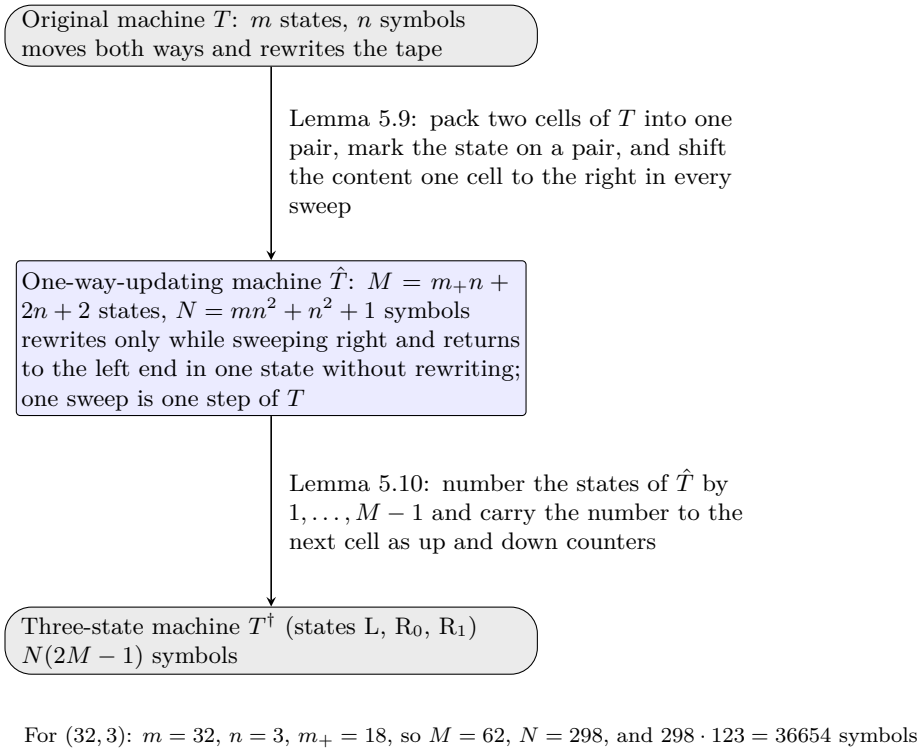


Figure 13: The two stages of Morita's conversion to three states

merged symbol, and ρ recovers the unmerged symbols on halting tapes; the hypothesis on the halting states prevents a run from continuing from a halting ID when merging makes the rule of another symbol applicable, and it holds because, in runs from encoded inputs, our block tables halt only by entering a halting state. Applied to the fixed tables of Table 3, merging removes one to three symbols. Much more is gained by choosing the blocks and values again, by simulated annealing, with the number of colors after merging as the objective. All five best results come from URTM$(13, 7)$ (Table 4). The two with four and five states were chosen again with the number of colors under the relaxed condition as the objective; under the condition before relaxing, the same tables need 43 and 40 colors. Their tables are not printed here; they are provided as text files, with the classes of merged symbols and the roles merged into each state, at `https://github.com/yokoyama-lab/small-machine-zoo/tree/main/tables`. We do not claim minimality, either of the coloring or over the choices of blocks.

3.4 Three states

Morita's conversion to three states [10, Sect. 5.3.5] has two stages (Fig. 13). Lemma 5.9 turns T into a *one-way-updating* RTM $\hat{T}$, which rewrites the tape only while sweeping to the right and returns to the left end in a single state without rewriting. To distinguish the blank of T from the blank of $\hat{T}$ and to extend the used region one cell at a time, its tape packs two cells of T into

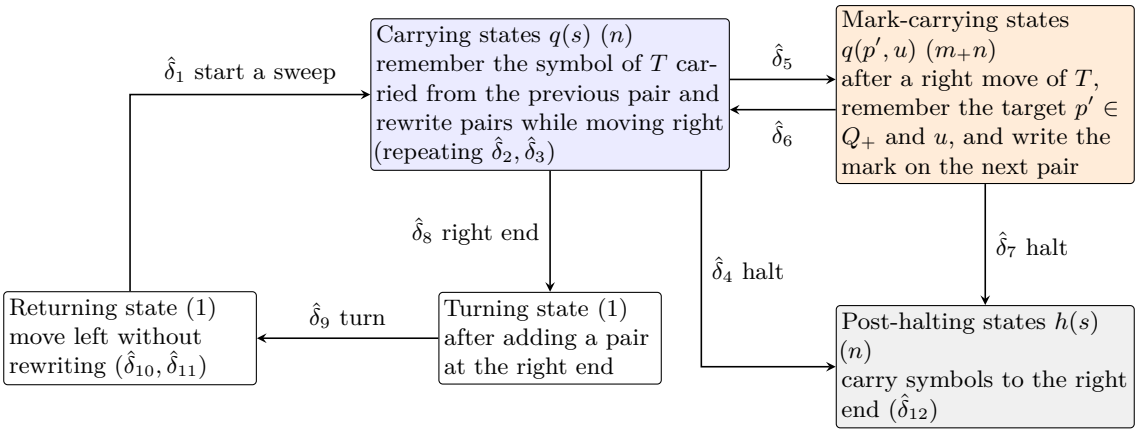


Figure 14: Kinds of states of $\hat{T}$ in Lemma 5.9 and the transitions between them

one cell. $\hat{T}$ has $N = mn^2 + n^2 + 1$ symbols and $M = m_+ n + 2n + 2$ states. The symbols are the blank of $\hat{T}$, the n^2 pairs $[s, t]$ holding the contents of two cells of T, and the mn^2 pairs $[[p, s], t]$ whose left component is the head cell of T and which are marked with the state p of T; thus $N = 1 + n^2 + mn^2$. The states are of four kinds, and $M = 1 + (n + 1) + m_+ n + n$.

- One state returns to the left end without rewriting.
- During the sweep to the right, n states remember the symbol of T carried from the previous cell, and one state turns back after adding a pair at the right end of the used region. Through this carrying, the symbols of T shift one place to the right in every sweep.
- After a right-moving quintuple of T is executed, $m_+ n$ states remember the target state $q \in Q_+$ and the carried symbol and go to write the mark on the pair in the next cell. The target of a left move can be written as a mark on the pair in the current cell, so these states are needed only for the states in Q_+.
- After T halts, n states carry symbols to the right end.

Figure 14 shows the transitions between these states (the rule names follow Morita). Lemma 5.10 then represents the state q_i of $\hat{T}$ by a counter $[i, x]$ with an up or a down mark on the symbol under the head and uses three states L, R_0, R_1, which gives $N(2M - 1)$ symbols. Figure 15 shows how it executes a right-moving quintuple of $\hat{T}$; as in Fig. 2 of Section 3.1, the number j of the state is carried to the right by counting down the down-marked counter in cell c while counting up the up-marked counter in cell $c + 1$, and the execution takes $2j + 1$ steps. For URTM$(32, 3)$ we have $m = 32$, $n = 3$, and $m_+ = 18$, so $N = 32 \cdot 3^2 + 3^2 + 1 = 298$ and $M = 18 \cdot 3 + 2 \cdot 3 + 2 = 62$; the result has $N(2M - 1) = 298 \cdot 123 = 36654$ symbols, which gives URTM$(3, 36654)$. Of these symbols, 85% are counters that combine a marked pair, one of the $m_+ n$ states of $\hat{T}$ entered after a right move, and an up or down mark. In a one-way-updating machine every carry goes to the right, so each state always reads a fixed side and the marks should not be needed; moreover, most counters are never read, and packing two cells into one creates the n^2 pairs. We reduce the count in four steps.

Shared counters. In a one-way-updating machine all carries go to the right, so each of the three states always reads counters on a fixed side: L only on the source side, R_0 and R_1 only on the target side. In Fig. 15, too, L reads only down-marked counters, and R_0 and R_1 read only up-marked ones. The same can be seen in Fig. 16, which shows all rules of Lemma 5.10 as a flowchart (the rule names follow Morita). Hence the up and down marks can be identified without losing determinism, and reversibility is preserved, since the written symbols of the rules

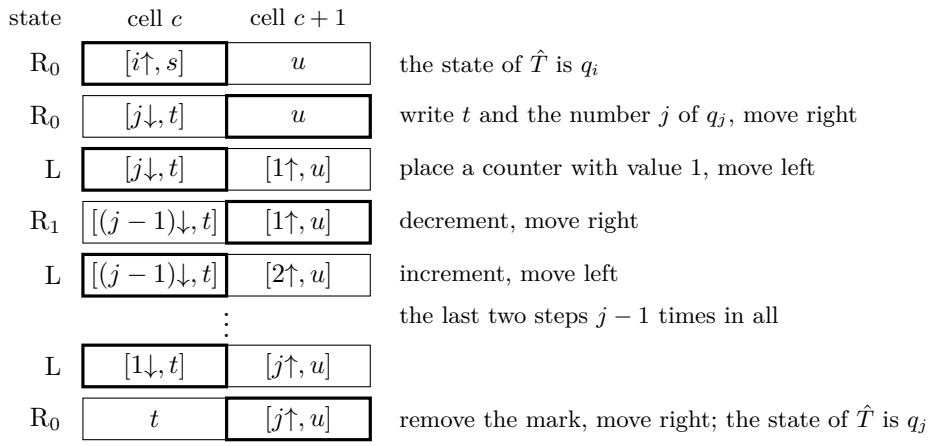


Figure 15: Morita's three-state machine executing a quintuple $[q_i, s, t, \mathrm{R}, q_j]$ of $\hat{T}$

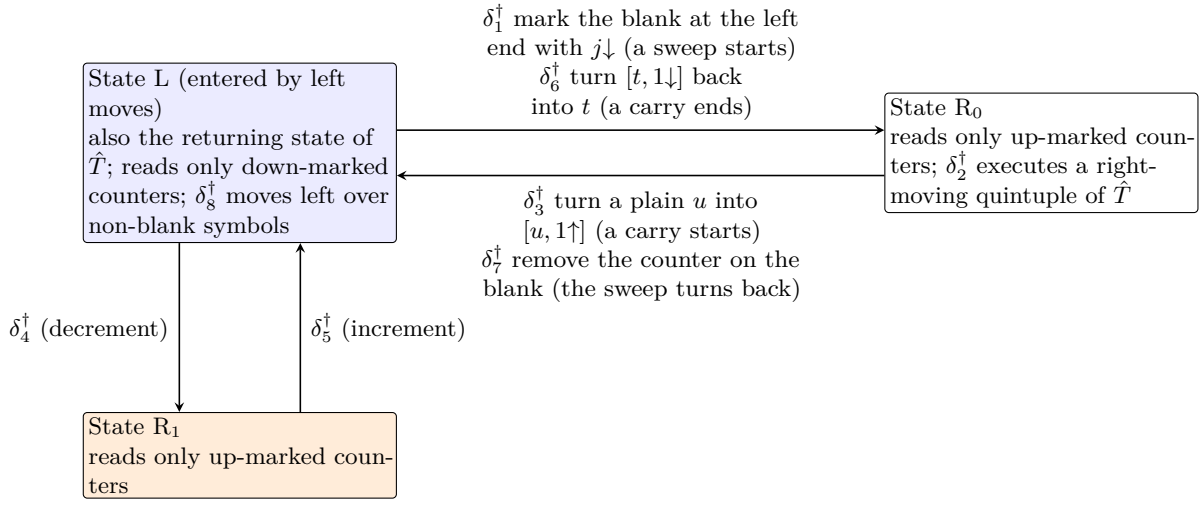


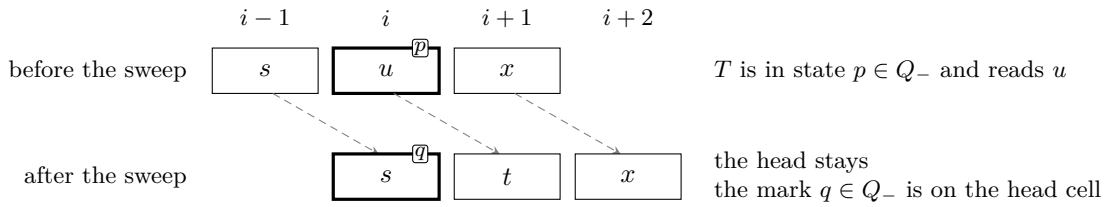
Figure 16: Flowchart of all rules of the three-state machine of Lemma 5.10

entering each state remain distinct. The count drops from $N(2M-1)$ to NM; for $(32, 3)$, where $N = 298$ and $M = 62$, it drops from $N(2M-1) = 36654$ to $NM = 298 \cdot 62 = 18476$.

Pruned counters. A counter $[k, x]$ is needed only if some state with number at least k can read x or is entered by writing x. Keeping only these counters gives $N + \sum_x \mathrm{need}(x)$ symbols, where $\mathrm{need}(x)$ is the largest such number. The sum depends on the numbering of the states of $\hat{T}$; states with the same set of relevant symbols can be kept adjacent, and the best order of these groups gives 1,820 symbols for $(32, 3)$. The machines with 18,476 and 1,820 symbols measure the effect of these two steps. For the one with 18,476 symbols we prove that it simulates every CTSH (but not universality in the sense of Definition 4); for the one with 1,820 symbols we only count its symbols and prove neither that it simulates T nor that the kept counters suffice along every run. The machines of the next two steps also share and prune counters, but their universality is proved directly for the concrete tables.

One symbol per cell. The remaining cost lies in the n^2 pairs and the mn^2 marked pairs. Instead of packing two cells, we keep one symbol per cell and represent the blank of T by a separate symbol B inside the used region; we write $\bar{T}$ for the one-way-updating machine obtained in this way. Since $\bar{T}$ also only sweeps to the right, the sweep would already have passed the target cell of a left move of T. As in Morita's construction, each sweep therefore shifts the whole

(a) left-moving quintuple $[p, u, t, \mathrm{L}, q]$



(b) right-moving quintuple $[p, u, t, \mathrm{R}, q]$

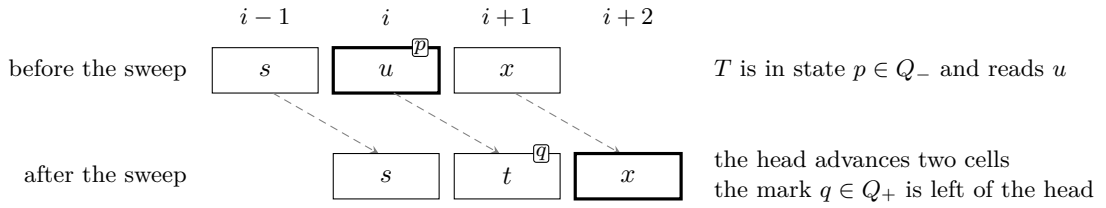


Figure 17: Execution of a quintuple of T by one sweep of $\bar{T}$

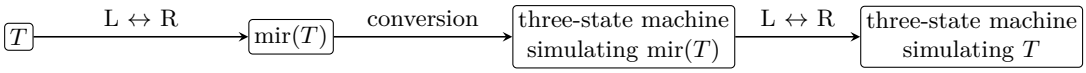
content of T one cell to the right: a left move of T then keeps the head where it is (the content of the target cell arrives at the current cell), and a right move advances it by two cells. The current state p of T is recorded as a mark on one cell: on the head cell if $p \in Q_-$, and on the cell to the left of the head if $p \in Q_+$. Since the states of Q_+ are entered only by right moves, a sweep that executes a right move can leave the written symbol and the target state together in the mark of one cell. Here, as in Morita's construction, a state that is not entered by any quintuple is put into Q_-. Figure 17 shows the tape before and after one sweep: the state name above a cell is its mark, the thick frame marks the head of T, and the dashed arrows show the shift by one cell to the right. If $p \in Q_+$, the mark before the sweep is on the cell to the left of the head instead, and the result of the sweep is the same. $\bar{T}$ has $n + 3 + |R| + |G|$ states, where R is the set of pairs (q, t) such that T has a right-moving quintuple into q writing t, and $G = R \cup (\{p_{\mathrm{init}}\} \times S)$ if the state p_{init} in which the run of T starts is in Q_+ and $G = R$ otherwise (S is the set of symbols of T). After the shared and pruned counters, the three-state machine T' has

$$1 + n + (m_- n + |G|)(n + 1) + n(n + 2 + |R| + |G|) + (n + 2)$$

symbols. Of these, $1 + n + (m_- n + |G|)$ are the symbols of $\bar{T}$ (the blank, the plain symbols, and the state marks), and the rest are counter symbols: n for each mark, $n + 2 + |R| + |G|$ for each plain symbol, and $n + 2$ for the blank. Since $|R| \le \min(|T|, m_+ n)$, where $|T|$ is the number of quintuples of T, this is $O(n(mn + |T|)) \subseteq O(mn^2)$, compared with $O(m\, m_+ n^3) \subseteq O(m^2 n^3)$ for Morita's construction. For $(32, 3)$ it gives 659 symbols.

Reflection. Applying the conversion to the mirror image $\mathrm{mir}(T)$ of T (directions L and R exchanged) and exchanging L and R back in the result gives a three-state machine that simulates T (Fig. 18(a)). In the mirror image, a right-moving quintuple $[p, s, t, \mathrm{R}, q]$ becomes the left-moving quintuple $[p, s, t, \mathrm{L}, q]$, so q moves from Q_+ to Q_- and the pair (q, t) leaves R. Conversely, a left-moving quintuple becomes right-moving, and its target enters Q_+ and its pair enters R (Fig. 18(b)). Thus the roles of m_+ and m_- and of the right-moving and left-moving targets are exchanged. In the symbol count, the coefficient of m_- is $n(n+1)$, that of $|G|$ is $2n+1$, and that of $|R|$ is n, so the exchange changes the count. For $(32, 3)$, T itself has $m_- = 14$ and $|R| = 46$, and $|G| = 47$ because the state in which T starts is in Q_+; this gives 659 symbols. The mirror image has $m_- = 18$ and $|R| = |G| = 36$ (Fig. 18(c)). The increase of m_- by 4 adds $4 \cdot 12 = 48$ symbols,

(a) The steps of reflection



(b) Quintuples and the side of their targets

quintuples of T		quintuples of mir(T)
$[p, s, t, \mathrm{R}, q]$: $q \in Q_+$, $(q, t) \in R$	⟶	$[p, s, t, \mathrm{L}, q]$: $q \in Q_-$, $(q, t) \notin R$
$[p', s', t', \mathrm{L}, q']$: $q' \in Q_-$	⟶	$[p', s', t', \mathrm{R}, q']$: $q' \in Q_+$, $(q', t') \in R$

(c) The values for $(32, 3)$

	m_+	m_-	$\lvert R\rvert$	$\lvert G\rvert$	symbols
T	18	14	46	47	659
mir(T)	14	18	36	36	600

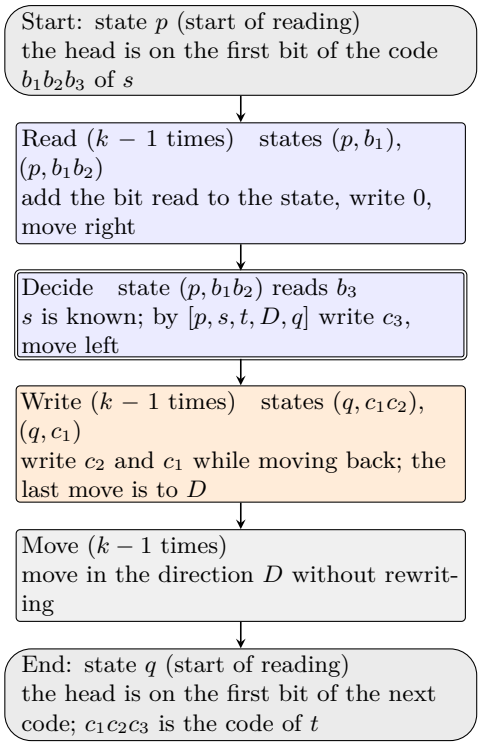
Figure 18: Reflection in the three-state conversion

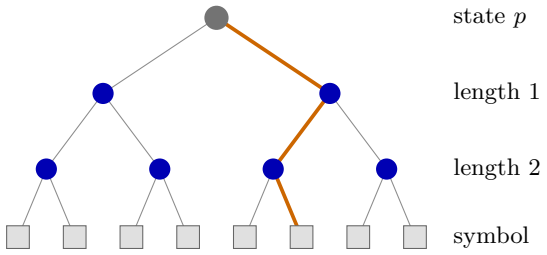
and the decreases of $|G|$ by 11 and of $|R|$ by 10 remove $77 + 30 = 107$, which gives 600 symbols, and hence URTM$(3, 600)$. Its table has 1,572 quintuples; instead of printing it, we provide it as a text file at `https://github.com/yokoyama-lab/small-machine-zoo/tree/main/tables`. Applied to the other five original machines, the improved conversion gives between 802 and 1,209 symbols, taking the better of the machine and its mirror image, so $(32, 3)$ is the best starting point.

3.5 Two symbols

Morita's conversion to two symbols [10, Sect. 5.3.3] replaces each symbol by a string of k bits, the blank by 0^k. A quintuple of T is simulated by reading $k - 1$ bits while erasing them, reading the last bit and writing the last bit of the new symbol, writing the remaining bits while moving back, and moving $k - 1$ cells in the direction of the move; the states remember the prefix read or the prefix still to write. If P_r and P_w denote the numbers of read and written prefixes of lengths 1 to $k - 1$, summed over the states, the number of states of T' is at most $mk + P_r + P_w$. Figure 19 shows the flowchart for $k = 3$. Among the shaded boxes, the states of the reading stage are counted in P_r and those of the writing stage in P_w, while the start state and the states of the moving stage number k for each state of T. The inner vertices of the tree on the right are the states that remember a read prefix. From URTM$(24, 4)$ $(k = 2)$ this gives URTM$(138, 2)$, and from URTM$(13, 7)$ $(k = 3)$ it gives 169 states. For $(24, 4)$, $P_r = P_w = 45$, so the number of states is $m' = mk + P_r + P_w = 24 \cdot 2 + 45 + 45 = 138$; for $(13, 7)$, $P_r = P_w = 65$, so it is $13 \cdot 3 + 65 + 65 = 169$.

The states exist to remember the prefix read so far and the prefix still to be written. Choosing the assignment of codes should let more symbols share prefixes, and skipping bits that need not be rewritten on the way back should remove auxiliary states. We add four degrees of freedom: the assignment of codes to symbols (the blank fixed to 0^k), the values written in place of the bits that have been read, a path type for each quintuple that says how many bits need not be rewritten on the way back (a right-moving quintuple can then cross its bit string once instead of three times), and, for each state, the end of the bit string at which the head stands when reading begins. Reversibility of the result reduces to distinct written values on the rules entering each auxiliary state. For fixed codes and starting ends, the written values and the path types are chosen by a constraint programming model that minimizes the number of auxiliary states; the codes and starting ends are chosen by exhaustive enumeration and hill climbing. Finally,





The left branch is 0 and the right branch 1; the thick path reads the code 101. The inner vertices are the states that remember a read prefix

Figure 19: Flowchart of Morita's conversion to two symbols ($k = 3$) and the tree of reading states

states whose rules agree on the entries defined for both are merged; a merged table fills undefined entries of one state with the rules of the other, so it simulates T only along runs that never halt in an undefined entry, which is the case for runs from encoded CTSH inputs. For $(13, 7)$, choosing only the codes gives 149 states, choosing also the written values and path types gives 135, and merging states gives 92. A randomized large-neighborhood search over the choices of the table before merging then gave a table with 137 states, more than 135 before merging but one that merges much further, down to 85 states and 153 quintuples, that is, URTM$(85, 2)$. Its table is provided at the same location as that of URTM$(3, 600)$. We have no closed formula for the size of this search result; it is verified for the concrete table. Applied to the other five original machines, the same method, including the search, gives 92, 92, 108, 115, and 115 states for $(10, 8)$, $(24, 4)$, $(15, 6)$, $(17, 5)$, and $(32, 3)$. These five tables are also proved universal in the sense of Definition 4, and all of them are dominated by URTM$(85, 2)$.

4 The new URTMs and their universality

Theorem 8 (Existence)**.** *For each of the 13 points*

$$(m, n) \in \{(3, 600), (4, 41), (5, 38), (6, 30), (7, 28), (8, 21), (10, 8),\\ (13, 7), (15, 6), (17, 5), (24, 4), (32, 3), (85, 2)\},$$

there is a table U that is deterministic and reversible, satisfies $\mathrm{NStates}(U) = m$ *and* $\mathrm{NSyms}(U) = n$*, and satisfies* $\mathrm{IsUTMStrict}(U)$.

These points are shown in Figure 1 (Section 1). The six points from $(10, 8)$ to $(32, 3)$ are Morita's original machines, which remain non-dominated; the other seven are new. The witnesses are concrete rule tables: Morita's distributed tables [12] and the tables produced from them by the conversions of Section 3.

Table 5 lists the numbers of symbols and states obtained by applying the conversions of Section 3 to each of Morita's six machines and to URTM$(85, 2)$. The columns of the conversions

Table 5: Results of the conversions applied to each URTM

	Four states (symbols)			Three states (symbols)		Two symbols (states)	
From	Morita	disjoint ranges	reverse counting	Morita	one symbol per cell	Morita	ours
$(10, 8)$	168	88*	48*	81075	1268 / 1209	139	92*
$(13, 7)$	189	98	61	98241	1073 / 1144	169	**85***
$(15, 6)$	186	96	53	77895	1038 / 935	174	108*
$(17, 5)$	175	90	55	55473	856 / 844	193	115*
$(24, 4)$	196	100	56	49323	832 / 802	138	92*
$(32, 3)$	195	99	57	36654	659* / **600***	182	115*
$(85, 2)$	342	172		68655	803 / 808		

that reduce states give numbers of symbols, and those of the conversions that reduce symbols give numbers of states. For each conversion, Morita's construction and ours are listed side by side. The column of Morita's four-state conversion gives the value of the formula $(2m + 1)n$; Morita gives the table only for $(10, 8)$. All other entries count the symbols or states of constructed tables. In the three-state column of our construction, the two values separated by "/" are for the machine itself and for its mirror image. Bold entries are points of Theorem 8, and the machines marked $*$ are proved universal in the sense of Definition 4. An empty entry was not computed (reverse counting for $(85, 2)$) or does not apply (a two-symbol conversion of a two-symbol machine). The results of the block decomposition and of symbol merging are in Tables 3 and 4.

4.1 Proof of universality

Each witness other than the five merged ones is first shown to simulate an arbitrary CTSH: for every CTSH $\hat{C}$ and initial string v, the machine started on the code of $(\hat{C}, v)$ halts if and only if $\hat{C}$ halts from $(v, 0)$, and its halting ID is the code of the halting ID of $\hat{C}$. For Morita's six machines this is a direct proof about the given tables. For the converted machines it follows from general simulation theorems for the conversions (preservation of determinism, reversibility, one-step simulation, and the correspondence of halting, halting IDs, and non-halting), instantiated with the concrete tables. The five merged machines inherit universality from their tables before merging by Theorem 7.

Universality in the sense of Definition 4 then requires computable maps between arbitrary TMs and CTSHs. We formalize the chain

$$\mathrm{tm}(i) \longrightarrow \text{two-register machine} \longrightarrow \text{2-tag system} \longrightarrow \text{CTSH} \longrightarrow U,$$

which follows the classical route of Cocke and Minsky [4] and Morita's Theorem 7.2. The first step stores the tape of $\mathrm{tm}(i)$ in two registers and normalizes the symbols of $\mathrm{tm}(i)$, which are arbitrary natural numbers, into binary; this replaces a separate conversion of the TM to two symbols. A general theorem then states that a machine that simulates an arbitrary CTSH and has a primitive recursive input encoding and a primitive recursive readout of the final string, independent of trailing blanks, satisfies IsUTMStrict. Each machine supplies only its assignment of natural numbers to states and symbols, its encoding of CTSHs, and its readout of final strings.

Proposition 9 (Agreement)**.** *Let U_1 and U_2 be deterministic tables satisfying* IsUTMStrict *with encodings c_1, c_2 and decodings d_1, d_2. For every TM* $\mathrm{tm}(i)$ *and every input a admitted in Definition 4, U_1 halts from* $\mathrm{toID}(0, c_1(i, a))$ *if and only if U_2 halts from* $\mathrm{toID}(0, c_2(i, a))$*, and if both halt, the decoded halting IDs are equal. In particular this holds for any two of the 13 witnesses of Theorem 8.*

The encoded inputs, the runs, and the numbers of steps differ between machines; only halting and the decoded result agree.

Table 6: Independent count of states and symbols in the witness tables

Table	Quint.	Sources	Target-only	Symbols	Zero-shifts
Morita's 6 points, distributed	59–84	m	2	n	2
same, tables in the book	57–82	m	0	n	0
5 symbol-merging points, distributed	143–162	m	2	n	2
same, without 2 quintuples	141–160	m	0	n	0
$(85, 2)$	153	85	0	2	0
$(3, 600)$	1,572	3	0	600	0

Halting quintuples. The distributed tables of Morita's six points represent the halting entries of the tables in the book by two zero-shift quintuples into halting states that are not counted, and the tables of the five points obtained by symbol merging inherit these two quintuples through the conversions; the tables of $(85, 2)$ and $(3, 600)$ have no halting quintuples. We prove Theorem 8 for these 11 points also for the tables without the two quintuples (for Morita's six points, these are the tables printed in the book). Removing them changes neither the number of states nor the number of symbols. A table without them halts one step before the original table enters a halting state, so the decoder first applies the removed quintuple once to the halting ID.

4.2 Counting states and symbols

Morita does not count halting states in the m of URTM(m, n). The book states that the set of final states is not specified in the problem of reducing the number of states, the state sets of the URTMs in Chapter 7 contain no halting states, and in Chapter 11 it notes that the halting states are not included in URTM$(17, 5)$ [10]. The distributed files also state explicitly that the halting states are not counted.

Our NStates counts source states and NSyms counts the symbols occurring in the table together with the blank. The two conventions could differ for states that occur only as targets, for halting states, and for symbols that do not occur in the table. For the witness tables of Theorem 8 we prove that every target is a source state; for $(85, 2)$ and $(3, 600)$ this holds already for the tables in the form of the distributed files, and for the other 11 points it holds for the tables without the two halting quintuples. Hence the number of states remains m even under a convention that also counts targets. We also prove that the blank occurs in each witness and that every symbol is both read and written, and that the witnesses have exactly (m, n) under Morita's way of counting (targets counted, only symbols occurring in the table). An independent count outside the proof assistant agrees (Table 6). For the 11 distributed tables with the additional halting quintuples, counting targets gives $m + 2$ states.

5 Nonexistence of URTMs with at most two states

Theorem 10 (Halting of two-state RTMs)**.** *There is a computable Boolean function f such that, for all distinct states q_0, q_1, q_f, every deterministic and reversible table T whose source states are among q_0, q_1 and whose targets are among q_0, q_1, q_f, every position $p \in \mathbb{Z}$, and every code $z \in Code$,*

$$f(T, p, z) = \text{true} \iff \text{Halts}_T(\text{toID}(p, z)).$$

Corollary 11. *If U is deterministic and reversible and* $\text{NStates}(U) \leq 2$*, then* $\neg\text{IsUTM}(U)$*, and hence* $\neg\text{IsUTMStrict}(U)$*. Moreover, there is a TM* $\text{tm}(i)$ *that is simulated, in the sense of conditions 2 and 3 of Definition 4, by no deterministic and reversible two-state table.*

The corollary follows from Theorem 10 and the undecidability of the halting problem. Two reductions bring a general table into the form of the theorem. A one-state table is a special case

of a two-state one. If a table has several halting targets, removing the quintuples whose target is not a source state does not change whether the machine halts: if the original run halts by executing a removed quintuple, the reduced run halts one step earlier. Removing quintuples preserves determinism and reversibility. The corollary therefore holds for any number of symbols, and with zero-shifts and halting states allowed as in Morita's definition. If this formalization captures Morita's Definition 7.1, it answers the question of the book and of the survey [11] negatively.

Proof outline of Theorem 10. Let T be as in the theorem, with the quintuples into q_f removed. Fix an initial finite ID and a window $[a, b]$ containing its non-blank cells and its head; let $w = b - a + 1$ and let n be the number of symbols.

1. *The state is the direction of the last move.* By the unique shift-direction property, if T has both a left-moving and a right-moving quintuple, then one state is entered only by left moves, the other only by right moves, and there is no zero-shift. So the state records the direction of the last move; we call the state entered by left moves L and the one entered by right moves R. T is then a partial injection $\pi : \{\mathrm{L}, \mathrm{R}\} \times S \rightharpoonup S \times \{\mathrm{L}, \mathrm{R}\}$, where S is the set of symbols of T. (If T moves in only one direction, say to the right, the argument is as follows. When the head first enters a blank cell $b+1, b+2, \ldots$ to the right of the window, everything to its right is blank and the head never moves left again, so the run from then on is determined by the state in which it enters. If the head reaches $b+3$, two of the three states in which it enters $b+1$, $b+2$, $b+3$ are equal, since there are only two states (Fig. 20). Let $c < c'$ be two cells entered in the same state. The run from c' is then the run from c shifted by $c' - c$ cells to the right, so the head keeps entering $c' + (c' - c)$, $c' + 2(c' - c)$, $\ldots$ in the same state and never halts. Hence the head of a halting run stays left of $b+3$, and a halting run is short. The case of moving only to the left is symmetric.)

2. *A half-line reports only that the head came back.* Fix a boundary and the half-line $\mathcal{H}$ to its right. The head always enters $\mathcal{H}$ in state R and leaves it in state L, and the behavior inside $\mathcal{H}$ does not depend on the content outside. Hence the outcome of the j-th entry into $\mathcal{H}$ (leave, halt inside, or never return) and the content of $\mathcal{H}$ on leaving depend only on the initial content of $\mathcal{H}$ and on j. Figure 21 illustrates this.

3. *The head cannot go deep into a blank half-line.* Let $[k, \infty)$ be initially blank and consider a halting run. The half-line $[k+1, \infty)$ is a translate of $[k, \infty)$, so by step 2 their contents at the j-th entry are translates of each other. Since, by step 2, the j-th entries into the two half-lines have the same outcome, if the number of entries into $[k+1, \infty)$ caught up with that into $[k, \infty)$, the run would repeat itself shifted by one cell and never halt. So the numbers of entries into $[k, \infty)$, $[k+1, \infty)$, $[k+2, \infty)$, $\ldots$ strictly decrease, and the head never passes k plus the number of entries into $[k, \infty)$.

4. *Each cell runs its own clock.* A cell to the right of the initial head is first entered from the left, and afterwards the head can return only from the side to which it left. So the pair (direction, symbol) at the i-th visit of the cell is obtained by iterating a fixed map $\kappa(d, s) = (\overline{d'}, v)$, where $\pi(d, s) = (v, d')$, on a set of size $2n$. All blank cells run the same clock, so a run that halts inside a blank half-line enters it at most $2n + 1$ times.

5. *Inside the window the machine is finite-state.* Replacing each step that leaves the window by a turn at its end gives a machine with at most $K = 3w\, n^w$ configurations, whose orbit the real run follows inside the window by step 2. A run that halts inside the window does so before the orbit becomes periodic; a run that halts in a half-line enters it in every period, hence by step 4 in boundedly many periods. In both cases each half-line is entered at most $K(2n + 3) + 1$ times.

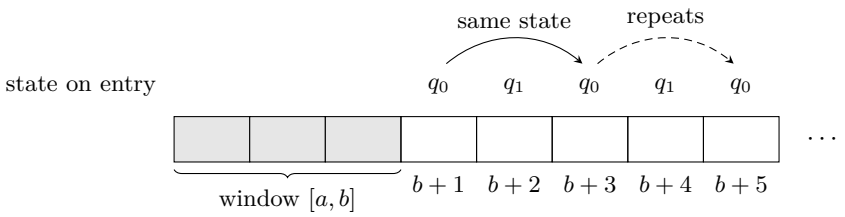


Figure 20: An example of the states in which a right-moving two-state table enters the blank cells to the right of the window

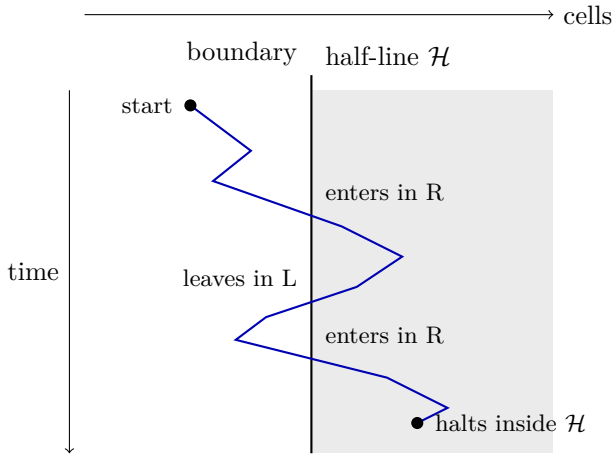


Figure 21: The head of a two-state RTM T crossing the boundary of a half-line

6. *A computable bound.* By steps 3 and 5 the head of a halting run stays in a computable interval around the window. A halting run of a deterministic machine never repeats an ID, so it halts within a number of steps bounded by a computable function of n and w. Running T for that many steps decides halting.

Only step 1 uses reversibility; steps 2 to 6 use only determinism and the fact that the state equals the direction of the last move. In an irreversible two-state TM the same state can be entered from both sides, so a half-line can send more than one bit of information, and indeed such machines can be universal. With three states, two states can be entered by right moves (as R_0 and R_1 in Morita's conversion to three states); the state in which the head enters a half-line then carries information, and step 2 fails.

Further nonexistence results.

Theorem 12. *If U is deterministic and* $\mathrm{NSyms}(U) \leq 1$*, then* $\neg\mathrm{IsUTM}(U)$*, even if U is not reversible.*

The tape never changes, so a halting run halts within a bounded number of steps.

Theorem 13. *Let U be deterministic and reversible with at most one left-moving quintuple ℓ, and assume that no zero-shift quintuple has the source state of ℓ as its target. Then* $\neg\mathrm{IsUTM}(U)$*. The same holds with right and left exchanged.*

This is a reversible counterpart of Pavlotskaya's theorem that the halting problem is decidable for two-symbol TMs with a single left-moving instruction [14]; here the number of states and symbols is arbitrary. By reversibility, the only quintuple entering the target of ℓ is ℓ itself.

6 Nonexistence of three-state two-symbol URTMs

Theorem 14. *Let U be deterministic and reversible, and suppose there are distinct states q_0, q_1, q_2 and a symbol $\sigma \neq 0$ such that every quintuple of U has its source state among q_0, q_1, q_2 and reads and writes only 0 and σ. Then* $\neg\mathrm{IsUTMStrict}(U)$.

The formalized proof first removes repeated quintuples from the table. This changes neither the steps nor halting, so it does not change IsUTMStrict either. Targets are arbitrary, so halting states and zero-shifts are allowed as in Morita's definition. We first prove the theorem for Morita's formulation on string IDs, with the additional requirement that the encoded inputs use only the symbols 0 and σ. For deterministic two-symbol tables, IsUTMStrict implies this formulation, because the only symbols that occur are the blank and one non-blank symbol; composing the two gives Theorem 14. As a check that the transfer is not vacuous, applying it to the witness for $(85, 2)$ shows that at least one machine satisfies the string-ID form.

Proof outline. Every table is first reduced to a normal form by renaming states and symbols and by removing the quintuples that enter a state other than q_0, q_1, q_2, which changes the halting time by at most one step. A normal form is determined by assigning to each of the six entries (three states, two symbols) either no quintuple or one of the six pairs of a written symbol and a target state, together with the direction in which each of the three states is entered; we call this a *key* (by reversibility, the assignments of the entries with quintuples are distinct). There are 359,829 keys. For each of them, a decision procedure checked by the kernel either produces a certificate or marks the table as exceptional. A certificate is a finite set of patterns of visits outside a window that is closed under the transitions of the table. It implies, by the pigeonhole principle, that a halting run halts within a computable number of steps, so that halting is decidable and the table is not universal. The exceptional keys are handled by general arguments:

- if a state has no entering quintuple, then after the first step the table is a two-state table, and Theorem 10 applies;
- if every quintuple entering some state comes from that state itself, then a halting run leaves that state within the width of the window plus one steps and never returns;
- if some state is entered by a zero-shift, then that quintuple can be composed with the next one, eliminating the state and again reducing to a two-state machine;

and the remaining keys by arguments for each type of behavior; the last of them are closed by showing that the head of a halting run stays within a bounded distance of the initial window. Counted outside the proof assistant up to renaming of states and left-right reflection, the exceptional keys form 178 classes of 2,136 tables, and the last group consists of 10 classes.

7 Formalization

All definitions of Section 2 and all theorems of Sections 3 to 6 are formalized in Lean 4 with Mathlib. Computability in Definition 4 is Mathlib's notion of computable functions. Every theorem is accepted by the Lean kernel, contains no `sorry`, and depends only on the standard axioms (propositional extensionality, the axiom of choice, and the soundness of quotients); in particular, all computations used in proofs are checked by the kernel. The formalization also proves formulas for the numbers of symbols of the conversions as functions of the original table: the block formula, $(2m + 1)n$ for Morita's four-state conversion, $(m + 1)n$ for disjoint ranges, $N(2M - 1)$ for Morita's three-state conversion, the formula of Section 3.4, and the state count of the k-bit two-symbol conversion; the reverse-counting expression is proved as an upper bound. It also proves the exact step counts of Morita's Lemmas 5.9 and 5.10.

Validation against published runs. Morita distributes description files with the rule tables and test runs of his nine URTMs and a Prolog simulator [12]. The formalized machine model reproduces, by kernel computation, the halting step counts of all published test runs. This includes the three runs of URTM$(3, 36654)$, a table with 37,936 quintuples, which take 471,798, 2,348,492, and 57,220 steps; these counts are obtained both by running the table and, independently, from the run of $(32, 3)$ and the exact step counts of Lemmas 5.9 and 5.10, and the kernel checks that the two routes prove the same statement. One published count differs: a test for URTM$(13, 7)$ states 1,073 steps, whereas recomputation gives 1,173. The book states that URTM$(13, 7)$ and URTM$(10, 8)$ take the same time to simulate a given CTSH, and the same test in the distributed file of URTM$(10, 8)$ also takes 1,173 steps. A simulator and a checker implemented independently of Lean agree with the formalized model on the tested runs.

Trusted base. The kernel guarantees that the proofs are correct relative to the definitions. What remains to be trusted is that the definitions and theorem statements express the intended claims, in particular that Definition 4 faithfully captures Morita's Definition 7.1; this correspondence cannot itself be checked mechanically. We have stated the definitions and the theorems in full in this paper so that the reader can compare them with the original. A further safeguard is that the same definitions are used on both sides: the existence results and the nonexistence results refer to the same notion of universality, so a definition that were trivially true or trivially false for all machines would contradict one of them. The Lean formalization and a full version, which includes further construction details and proofs, are available at `https://github.com/yokoyama-lab/small-machine-zoo`.

8 Conclusion

We have constructed seven new URTMs that, together with six of Morita's machines, give 13 non-dominated points, and we have shown that no URTM exists with at most two states or with three states and two symbols. The new points come from making Morita's conversions economical: disjoint ranges by direction, counting in reverse, splitting state numbers into blocks, sharing and pruning counters, and merging symbols. The two-state result rests on the observation that, in a reversible machine with two states, the state is determined by the direction of the last move.

Several questions remain open. None of the new points is known to be minimal, and the lower and upper bounds are far apart: no URTM with at most two states exists, while the smallest known three-state URTM has 600 symbols. It is open whether some three-state, three-symbol or four-state, two-symbol RTM is universal. For irreversible UTMs, the smallest known machines have far fewer symbols for the same number of states, and it would be interesting to know whether reversibility forces this difference.

Acknowledgments

We are grateful to the late Kenichi Morita, who introduced us to this problem. This work was supported by JST CREST Grant Number JPMJCR24I4, Japan.

A Construction details and intermediate counts

This appendix brings together the construction and proof explanations that accompany the short note. The Lean definitions and theorem statements determine the exact hypotheses and conclusions. The explanations below do not constitute a human inspection of the formalization.

A.1 Four-state and block constructions

Let an original machine have m states and n symbols. Write m_+ and m_- for the numbers of states entered by right and left moves, respectively. Morita's four-state conversion uses $(2m+1)n$ symbols. Giving the two directions separate number ranges reduces the expression to $(m+1)n$. Counting in reverse while carrying a symbol to the left gives the upper bound $(\max(m_+, m_-)+1)n$. Splitting states into blocks, merging roles, and removing unused symbol–counter pairs gives $n + \sum_{i \in V} |X_i|$, where V is the set of counter values used and X_i is the set of original symbols used with value i. For the conversion from $(10, 8)$, the respective symbol counts are 168, 88, 48, and 46. The last value is the block construction before the later symbol-merging improvements. The 88-, 48-, and 46-symbol tables are each proved universal in Lean.

The corresponding formal statements are collected in `LeanUniv.FinalFormulas`. The expression for reverse counting is an upper bound; the other formulas just listed are equalities for the stated conversion functions. These formulas concern construction families and do not give lower bounds over all universal machines.

For the final five symbol-merging points, the original machine is $(13, 7)$. The recipe files specify the blocks and counter choices, and the merge files specify classes of symbols. The generators reconstruct the tables, while Lean checks their finite properties and applies the general symbol-merging theorem. Rebuilding these files reproduces the selected constructions; it does not repeat the optimization search or establish optimality.

A.2 Three-state conversion

For Morita's three-state conversion, let $M = m_+n + 2n + 2$ and $N = mn^2 + n^2 + 1$ be the sizes of the intermediate machine. The original symbol-count formula is $N(2M - 1)$. Sharing the up and down marks gives NM. A further construction assigns a separate counter limit to each symbol; its count is N plus the sum of these limits.

For the source $(32, 3)$, the successive recorded symbol counts are 36,654, 18,476, 1,820, 659, and 600. The 1,820-symbol intermediate table has only a size calculation: adequacy of its counter limits throughout every run was not proved. The 659-symbol table rebuilds the intermediate machine using one symbol per cell. Reflecting the source machine left to right gives the 600-symbol table. Simulation and universality of the 659-symbol table and of the final $(3, 600)$ witness are established by the corresponding Lean theorems.

A.3 Two-symbol conversion and automated search

For a conversion using k-bit codes, let P_r and P_w be the sums, over states, of the numbers of read-code and write-code prefixes of lengths $1, \ldots, k-1$. Let D_0 be the set of source states having a rule whose target is also a source state. The formalized state-count formula is $|D_0| + (k-1)m + P_r + P_w$, bounded above by $mk + P_r + P_w$.

From $(13, 7)$, the recorded state counts are 169 using Morita's code, 149 with a search-selected code, 135 after selecting written values and path types, 92 after state merging, and 85 after the broader automated search. The solver that selected the intermediate tables is not itself a verified Lean conversion function. Its concrete output tables are checked in Lean; the proof for the merged table also establishes the required relationship to its pre-merge table. The relevant intermediate-value theorems are in `LeanUniv.FinalValues`.

The automated search was modeled on Karpathy's autoresearch (`https://github.com/karpathy/autoresearch`). An interactive AI session prepared the search program, fixed evaluator, and instructions at the human's request. After the human started a session, the agent edited the search program and the evaluator checked candidates for determinism, reversibility, and finite CTSH simulations. Only candidates subsequently proved in Lean counted as results. The search found $(6, 37)$, $(8, 28)$, $(9, 27)$, and $(85, 2)$; only the last remains non-dominated after

the symbol-merging improvements. The repository includes the construction recipes needed to rebuild the final witnesses.

B Outline of the two-state decidability proof

The formal entry points are `halting_decidable_uniform` and `no_le_two_state_URTM`. This section summarizes the structure of the existing proof; the formal statements specify its scope.

First remove transitions into states with no outgoing transitions by the reduction described in the main text. Let $[a, b]$ contain the initial head position and all initially non-blank cells, and put $w = b - a + 1$. Let Σ contain the blank and every symbol read or written by the table, and let $n = |\Sigma|$. Consider an initial ID whose tape symbols lie in Σ.

1. In a reversible table, transitions entering the same state move in the same direction. If a two-state table has both left and right moves, one state is entered only from the left and the other only from the right; there is no remaining state for a zero-shift target. The state therefore records the direction of the last move. The transition function can be represented by a partial injection on direction–symbol pairs.

2. Consider a half-line to the right of a fixed boundary. Every entry has the right-moving state and every exit the left-moving state. While the head is outside the half-line, its contents do not change. The outcome of the next entry therefore depends on its initial contents and on the number of earlier entries, rather than on the outside tape. The only information passed back to the outside is the return itself.

3. For initially blank nested half-lines, their local computations are translations of one another. In a halting computation, each visited deeper half-line has fewer entries than its predecessor. Otherwise the computation repeats translated to the right and never halts. This bounds the maximum depth by the number of entries at the outer boundary.

4. The sequence of visits to a cell follows a finite local clock on direction–symbol pairs. There are $2n$ such pairs. This bounds by $2n + 1$ the number of entries into a blank half-line if the computation halts inside it. The left-hand case follows by reflection.

5. Replace excursions outside $[a, b]$ by immediate returns at its ends. The resulting dynamics within the window is finite. In the existing proof, $Z = n^w \cdot 3w$ bounds the window states, and $N = Z(2n + 3) + 1$ bounds the relevant entries into each outside half-line. Combining this with the depth bound confines a halting run to $[a - N, b + N]$.

6. That interval contains finitely many IDs over the finite alphabet. A halting run of a deterministic machine cannot repeat an ID. Thus a computable bound on the number of steps suffices to decide halting by bounded simulation. The bound need not be practical.

If only one movement direction occurs, once the head leaves the initial window it repeatedly reads blanks. Repetition of a state gives a translated repetition of the run. A halting run can therefore be bounded in this case as well. The general nonexistence theorem combines the computability of halting for these tables with Mathlib's undecidability of the halting problem.

The first step is where reversibility restricts the two-state model. With three states, a half-line can receive different kinds of entries moving in the same direction; the two-state argument does not apply unchanged.

C Outline of the three-state two-symbol proof

The proof normalizes the tables and checks the 359,829 keys of the normal forms with the Lean kernel. It handles most tables by certificates constraining visits outside a window and

by finite-state bounds. Other cases reduce to bounded halting for a two-state machine after removing a state, or require a separate argument about their behavior.

Counts of equivalence classes up to renaming and left–right reflection were computed outside Lean: there are 26,861 such classes in the recorded enumeration, and the last ten classes contain 120 tables. The AI supplied arguments for these final classes, proving that a halting run stays within a bounded neighborhood of the initial window.

The resulting theorem is `URTM.Meta.no_three_two_URTM_all`, followed by the transfer to `no_three_two_URTMStrict_all`. The original target condition is the string-ID form of universality with the stated restriction on encoded symbols. The transfer from `IsUTMStrict` is essential: a proof of nonexistence for a different universality definition alone would not imply the claim made in the note. The finite enumeration files and all imported proof dependencies are included in the repository.

D Comparison with prior work and verification scope

Morita's nine machines provide the baseline [10]. Six of these points remain on the frontier together with the seven new points. The work improves conversion and symbol-assignment choices; it does not prove that the resulting state–symbol pairs are globally smallest. Kudlek and Rogozhin's merging of a blank and another symbol in an ordinary UTM [7] guided the symbol-merging stage. The general theorem here makes the preservation conditions explicit for the formalized universality predicate, including recovery of the symbols on halting tapes.

The lower-bound region quoted for standard UTMs comes from the survey by Woods and Neary [16]. Its conventions for halting and zero-shifts differ from the present model. The new nonexistence claims are therefore established directly in the stated reversible model rather than inferred solely from that comparison. The AI also used prior work on irreversible machines to guide the three-state proof; the author has not checked that correspondence against the original sources.

The supplied generators read Morita's original test configurations from the machine descriptions attached to his 2015 slides. For eight machines, Lean reproduces 44 published halting runs, with the count 1,073 in one $(13, 7)$ comment corrected to 1,173. For $(3, 36654)$, the exact step counts are 471,798, 2,348,492, and 57,220. The large table is checked through a lookup function proved to agree with it and certificates for successive run segments. These finite examples support comparison of the operational model with the supplied data, while the general simulation and universality theorems cover arbitrary encoded inputs.

The two proof environments reuse the specification by copying it. The repository checks that the copies of `Spec.lean` are byte-identical and that the copies of `Strict.lean` differ only in their imports. Each environment is built separately and its imported declarations are audited for nonstandard axioms. These checks establish properties of the formal artifacts; they do not establish the faithfulness of the formalization to the intended mathematical claims.

References

[1] A. Asperti and W. Ricciotti. A formalization of multi-tape Turing machines. *Theoretical Computer Science*, 603:23–42, 2015. `https://doi.org/10.1016/j.tcs.2015.07.013`.

[2] C. H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. `https://doi.org/10.1147/rd.176.0525`.

[3] M. Carneiro. Formalizing computability theory via partial recursive functions. In *10th International Conference on Interactive Theorem Proving (ITP 2019)*, LIPIcs 141, pages 12:1–12:17, 2019. `https://doi.org/10.4230/LIPIcs.ITP.2019.12`.

[4] J. Cocke and M. Minsky. Universality of tag systems with $P = 2$. *Journal of the ACM*, 11(1):15–20, 1964. https://doi.org/10.1145/321203.321206.

[5] A. Dudenhefner. Certified decision procedures for two-counter machines. In *7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022)*, LIPIcs 228, pages 16:1–16:18, 2022. https://doi.org/10.4230/LIPIcs.FSCD.2022.16.

[6] Y. Forster, F. Kunze, and M. Wuttke. Verified programming of Turing machines in Coq. In *Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 114–128, 2020. https://doi.org/10.1145/3372885.3373816.

[7] M. Kudlek and Y. Rogozhin. A universal Turing machine with 3 states and 9 symbols. In *Developments in Language Theory*, Lecture Notes in Computer Science 2295, pages 311–318. Springer, 2002. https://doi.org/10.1007/3-540-46011-X_27.

[8] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science 12699, pages 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[9] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. https://doi.org/10.1145/3372885.3373824.

[10] K. Morita. *Theory of Reversible Computing*. Springer, 2017. https://doi.org/10.1007/978-4-431-56606-9.

[11] K. Morita. Making reversible computing machines in a reversible cellular space. *Bulletin of the EATCS*, no. 140, 2023. https://bulletin.eatcs.org/index.php/beatcs/article/view/760.

[12] K. Morita. Constructing small universal reversible Turing machines (slides and description files). Hiroshima University Institutional Repository. https://hiroshima.repo.nii.ac.jp/records/2000034.

[13] T. Neary and D. Woods. Four small universal Turing machines. *Fundamenta Informaticae*, 91(1):123–144, 2009. https://doi.org/10.3233/FI-2009-0036.

[14] L. M. Pavlotskaya. Solvability of the halting problem for certain classes of Turing machines. *Mathematical Notes of the Academy of Sciences of the USSR*, 13(6):537–541, 1973. https://doi.org/10.1007/BF01163965.

[15] Y. Rogozhin. Small universal Turing machines. *Theoretical Computer Science*, 168(2):215–240, 1996. https://doi.org/10.1016/S0304-3975(96)00077-1.

[16] D. Woods and T. Neary. The complexity of small universal Turing machines: A survey. *Theoretical Computer Science*, 410(4–5):443–450, 2009. https://doi.org/10.1016/j.tcs.2008.09.051.

[17] J. Xu, X. Zhang, and C. Urban. Mechanising Turing machines and computability theory in Isabelle/HOL. In *Interactive Theorem Proving (ITP 2013)*, Lecture Notes in Computer Science 7998, pages 147–162. Springer, 2013. https://doi.org/10.1007/978-3-642-39634-2_13.